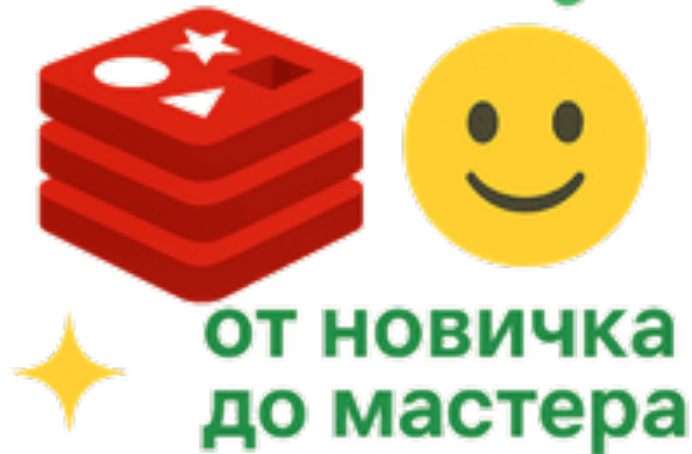


Lua-скрипты в Redis



ОТ НОВИЧКА
ДО МАСТЕРА

Lua-программист для разработки с Redis в реальном проекте

Модуль 1 - Введение

Урок 1 - Что такое Redis

Урок 2 - Зачем нужны Lua-скрипты в Redis

Урок 3 - Преимущества Lua-скриптов для атомарных операций и сложной логики

Урок 4 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты только что устроился в команду, которая использует Redis для хранения игровых данных. Тебе нужно понять, что такое Redis.

Подсказка: Подумай о Redis как о сверхбыстрой записной книжке для программ.

Ответ:

- **Redis** — это быстрый ключ-значение хранилище данных в памяти
- **Redis** — это язык программирования
- **Redis** — это графический редактор
- **Redis** — это облачный сервис для видеозвонков

🧙 Шаг 02:

Миссия 2

Твой наставник говорит, что Lua-скрипты помогут управлять данными в Redis эффективнее. Что это значит?

Подсказка: Представь, что вместо ручного выполнения нескольких шагов ты создаёшь волшебную команду для Redis.

Ответ:

- **Lua-скрипты позволяют объединять несколько операций в одну атомарную команду**
- Lua-скрипты заменяют язык Redis на новый
- Lua-скрипты нужны только для графического интерфейса
- Lua-скрипты автоматически ускоряют интернет-соединение

🧙 Шаг 03:

Миссия 3

Ты заметил, что некоторые операции с данными должны выполняться одновременно, чтобы избежать ошибок. Как Lua-скрипты помогают?

Подсказка: Подумай о ситуации, когда несколько игроков одновременно тянут один сундук с золотом.

Ответ:

- **Lua-скрипты обеспечивают атомарность операций, предотвращая конфликты**
- Lua-скрипты разделяют операции на отдельные потоки
- Lua-скрипты ускоряют работу процессора
- Lua-скрипты создают резервные копии данных каждые секунды

🧙 Шаг 04:

Миссия 4

Сопоставь ключевые понятия Redis с их объяснением.

Подсказка: Вспомни, как разные магические предметы в игре выполняют уникальные функции.

Ответ:

- **Ключ-значение** — Форма хранения данных в Redis, где ключ уникален
- **Атомарная операция** — Операция, которая выполняется полностью или не выполняется вовсе
- **Lua-скрипт** — Скрипт, позволяющий объединять несколько операций в одну команду
- **In-memory хранилище** — Данные хранятся в памяти для быстрой работы

🧙 Шаг 05:

Миссия 5

Сопоставь преимущества Lua-скриптов с их результатами в игре.

Подсказка: Подумай, что делает жизнь программиста проще и данные безопаснее.

Ответ:

- **Атомарные операции** — Игроки не смогут получить дубликаты предметов
- **Сложная логика внутри скрипта** — Множество действий выполняются одним нажатием кнопки
- **Высокая скорость исполнения** — Игровые события происходят без задержек

- **Меньше ошибок при одновременных действиях** — Система автоматически предотвращает конфликт изменений

🧙 Шаг 06:

Миссия 6

Сопоставь термины Lua с их значением в Redis.

Подсказка: Lua — это как заклинание: каждая строчка имеет своё действие.

Ответ:

- **redis.call** — Выполнение команды Redis внутри скрипта
- **return** — Возврат значения из Lua-скрипта в Redis
- **ARGV** — Список аргументов, переданных скрипту
- **KEYS** — Список ключей Redis, используемых в скрипте

🧙 Шаг 07:

Миссия 7

Расположи шаги использования Lua-скрипта в Redis в правильном порядке.

Подсказка: Подумай, как герой в игре планирует миссию: подготовка, выполнение и результат.

1. Определить, какие ключи и аргументы нужны
2. Написать Lua-скрипт с нужной логикой
3. Выполнить скрипт в Redis через EVAL или EVALSHA
4. Получить результат и обработать его

Ответ:

- Определить, какие ключи и аргументы нужны
- Написать Lua-скрипт с нужной логикой
- Выполнить скрипт в Redis через EVAL или EVALSHA
- Получить результат и обработать его

🧙 Шаг 08:

Миссия 8

Упорядочи действия программиста при необходимости объединить несколько операций в одну атомарную.

Подсказка: Думай как стратег: сначала план, потом действия, потом проверка.

1. Определить все операции, которые должны быть атомарными
2. Создать Lua-скрипт, объединяющий эти операции
3. Запустить скрипт и проверить результат
4. Использовать скрипт в рабочем процессе для предотвращения конфликтов

Ответ:

- Определить все операции, которые должны быть атомарными
- Создать Lua-скрипт, объединяющий эти операции
- Запустить скрипт и проверить результат
- Использовать скрипт в рабочем процессе для предотвращения конфликтов

🧙 Шаг 09:

Миссия 9

Расположи этапы отладки Lua-скрипта в Redis по логике успешной миссии.

Подсказка: Как в игре: сначала проверяешь карту, потом пробуешь действие, затем исправляешь ошибки.

1. Проверить синтаксис скрипта
2. Выполнить скрипт на тестовом наборе данных
3. Проанализировать ошибки и логи
4. Внести исправления и повторить проверку

Ответ:

- Проверить синтаксис скрипта
- Выполнить скрипт на тестовом наборе данных
- Проанализировать ошибки и логи
- Внести исправления и повторить проверку

Шаг 10:

Миссия 10

Расставь шаги, чтобы полностью автоматизировать обновление игровой статистики с помощью Lua-скрипта в Redis.

Подсказка: Думай как командир операции: подготовка, выполнение, проверка, использование.

1. Определить ключи, хранящие статистику игроков
2. Написать Lua-скрипт для обновления статистики атомарно
3. Протестировать скрипт на тестовых данных
4. Внедрить скрипт в игровую систему и следить за результатом

Ответ:

- Определить ключи, хранящие статистику игроков
- Написать Lua-скрипт для обновления статистики атомарно
- Протестировать скрипт на тестовых данных
- Внедрить скрипт в игровую систему и следить за результатом

Урок 5 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты только что присоединился к команде разработчиков и впервые сталкиваешься с Redis. Что из нижеперечисленного лучше всего описывает Redis?

Подсказка: Подумай о том, как программисты быстро хранят и получают данные.

Ответ:

- **Быстрая база данных ключ-значение в памяти**
- Язык программирования для веб-приложений
- Система управления версиями
- Фреймворк для мобильных приложений

Шаг 02:

Миссия 2

Ты заметил, что команда использует Lua-скрипты в Redis. Зачем они нужны?

Подсказка: Подумай о том, как можно выполнять несколько действий с базой данных одновременно.

Ответ:

- **Для выполнения атомарных операций и сложной логики внутри Redis**
- Чтобы заменить SQL-запросы
- Для генерации веб-страниц
- Для хранения больших файлов на диске

Шаг 03:

Миссия 3

Ты планируешь использовать Lua-скрипт для обновления нескольких ключей сразу. Какое преимущество ты получаешь?

Подсказка: Подумай о том, что может пойти не так, если операции не атомарные.

Ответ:

- **Все изменения выполняются как единое целое без частичных обновлений**
- Можно выполнять изменения только по одному ключу
- Lua автоматически ускоряет выполнение запросов на сервере
- Можно использовать Lua для создания графического интерфейса

Шаг 04:

Миссия 4

Формулировка: Сопоставьте тип операции в Redis с её примером использования.

Понятия:

Ответ:

- GET
- SET
- INCR
- DEL

Определения:

1. Получить значение по ключу
2. Удалить ключ из базы данных
3. Установить значение для ключа
4. Увеличить числовое значение на 1

Подсказка: Вспомни, какие базовые команды нужны для работы с ключами и значениями.

- **GET** — Получить значение по ключу
- **SET** — Установить значение для ключа
- **INCR** — Увеличить числовое значение на 1
- **DEL** — Удалить ключ из базы данных

Шаг 05:

Миссия 5

Формулировка: Сопоставьте преимущества Lua-скриптов с их описанием.

Понятия:

Ответ:

- Атомарность
- Скорость выполнения
- Сложная логика

Определения:

1. Все действия выполняются как единое целое
2. Выполнение операций быстрее, чем через клиентский код
3. Можно выполнять условные и циклические операции внутри Redis

Подсказка: Подумай, почему программисты любят Lua для сложных последовательностей действий.

- **Атомарность** — Все действия выполняются как единое целое
- **Скорость выполнения** — Выполнение операций быстрее, чем через клиентский код
- **Сложная логика** — Можно выполнять условные и циклические операции внутри Redis

 Шаг 06:

Миссия 6

Формулировка: Сопоставьте сценарий использования Lua-скрипта с его преимуществом.

Понятия:

Ответ:

- Обновление нескольких счетчиков одновременно
- Проверка условий перед записью значения
- Очистка ключей по шаблону

Определения:

1. Позволяет гарантировать целостность данных
2. Уменьшает количество сетевых обращений к Redis
3. Автоматизация рутинных задач на сервере

Подсказка: Подумай, какие задачи Lua позволяет делать на сервере без лишних запросов.

- **Обновление нескольких счетчиков одновременно** — Позволяет гарантировать целостность данных
- **Проверка условий перед записью значения** — Уменьшает количество сетевых обращений к Redis
- **Очистка ключей по шаблону** — Автоматизация рутинных задач на сервере

 Шаг 07:

Миссия 7

Вопрос: Расположите шаги написания Lua-скрипта в Redis в правильном порядке:

1. Отправка скрипта на сервер Redis
2. Определение логики и команд, которые нужно выполнить
3. Проверка работы скрипта в тестовой среде
4. Использование скрипта в реальном приложении

Подсказка: Сначала пишем, потом тестируем и только потом используем в боевом режиме.

Ответ:

- Определение логики и команд, которые нужно выполнить
- Отправка скрипта на сервер Redis
- Проверка работы скрипта в тестовой среде
- Использование скрипта в реальном приложении

 Шаг 08:

Миссия 8

Вопрос: Расположите процесс отладки Lua-скрипта в правильной последовательности:

1. Локальное тестирование скрипта
2. Логирование значений внутри скрипта
3. Использование скрипта на тестовом сервере Redis
4. Анализ ошибок и исправление логики

Подсказка: Отладка похожа на игру: сначала пробуем, затем смотрим, что пошло не так, и исправляем.

Ответ:

- Локальное тестирование скрипта
- Логирование значений внутри скрипта
- Использование скрипта на тестовом сервере Redis
- Анализ ошибок и исправление логики

 Шаг 09:

Миссия 9

Вопрос: Расположите этапы интеграции Lua-скрипта в реальное приложение:

1. Создание резервной копии данных Redis
2. Внедрение скрипта в рабочий код
3. Наблюдение за поведением приложения и производительностью
4. Корректировка скрипта при необходимости

Подсказка: Без безопасности и резервного копирования любые эксперименты опасны.

Ответ:

- Создание резервной копии данных Redis
- Внедрение скрипта в рабочий код
- Наблюдение за поведением приложения и производительностью
- Корректировка скрипта при необходимости

 Шаг 10:

Миссия 10

Вопрос: Расположите действия при оптимизации Lua-скрипта в Redis:

1. Профилирование выполнения скрипта
2. Определение узких мест и затратных операций
3. Переписывание или улучшение проблемных частей
4. Повторное тестирование и проверка производительности

Подсказка: Оптимизация — это игра с ресурсами: сначала измеряем, потом исправляем и проверяем.

Ответ:

- Профилирование выполнения скрипта
- Определение узких мест и затратных операций
- Переписывание или улучшение проблемных частей
- Повторное тестирование и проверка производительности

Модуль 2 - Основы Lua

 **Урок 1 - Краткий обзор языка Lua (типизация, базовые конструкции, функции)**

 **Урок 2 - Установка и запуск Lua интерпретатора (для практики)**

 **Урок 3 - Как писать и тестировать простые Lua-скрипты**

 **Урок 4 - Квест по мотивам реальной работы программистов — 1**

 **Шаг 01:**

Миссия 1

Ты впервые открываешь Lua интерпретатор. Какая особенность Lua отличает его от других языков?

Подсказка: Подумай о гибкости и простоте типов данных.

Ответ:

- **Динамическая типизация и простая структура языка**
- Жесткая типизация и сложные синтаксические конструкции
- Компиляция в машинный код без интерпретатора
- Использование только объектно-ориентированного программирования

 **Шаг 02:**

Миссия 2

Ты хочешь написать первый Lua-скрипт. Какой способ запуска скрипта самый простой для практики?

Подсказка: Подумай о том, как проверить код мгновенно без сложной настройки.

Ответ:

- **Использовать Lua интерпретатор напрямую в командной строке**
- Компилировать скрипт в исполняемый файл
- Запускать скрипт через веб-браузер
- Запускать только в IDE без интерпретатора

 **Шаг 03:**

Миссия 3

Ты пишешь скрипт, который считает сумму чисел. Какой элемент Lua поможет организовать повторяющиеся действия?

Подсказка: Подумай о том, как автоматизировать повторение действий в коде.

Ответ:

- **Циклы (for, while)**
- Комментарии
- Переменные глобального уровня
- Только if-условия

Шаг 04:

Миссия 4

Формулировка: Сопоставьте базовые конструкции Lua с их описанием.

Понятия:

Ответ:

- if
- for
- function
- local

Определения:

1. Объявление функции
2. Локальная переменная с областью видимости внутри блока
3. Цикл с фиксированным числом итераций
4. Условная проверка

Подсказка: Подумай, какие конструкции управляют потоком кода, а какие — создают функции или переменные.

- **if** — Условная проверка
- **for** — Цикл с фиксированным числом итераций
- **function** — Объявление функции
- **local** — Локальная переменная с областью видимости внутри блока

Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы данных Lua с примером.

Понятия:

Ответ:

- number
- string
- boolean
- table

Определения:

1. Истина или ложь
2. Последовательность символов
3. Коллекция значений, ассоциативный массив
4. Число, целое или с плавающей точкой

Подсказка: Подумай о том, что хранит числа, тексты, коллекции и логические значения.

- **number** — Число, целое или с плавающей точкой

- **string** — Последовательность символов
- **boolean** — Истина или ложь
- **table** — Коллекция значений, ассоциативный массив

Шаг 06:

Миссия 7

Вопрос: Расположите шаги запуска Lua-скрипта в правильном порядке:

1. Написание скрипта
2. Открытие Lua интерпретатора
3. Ввод команды запуска скрипта
4. Наблюдение результата выполнения

Подсказка: Сначала создаем код, потом запускаем, затем смотрим, что получилось.

Ответ:

- Написание скрипта
- Открытие Lua интерпретатора
- Ввод команды запуска скрипта
- Наблюдение результата выполнения

Шаг 07:

Миссия 8

Вопрос: Расположите шаги отладки простого Lua-скрипта:

1. Добавление print() для проверки значений
2. Исправление ошибок, если результат неправильный
3. Повторное выполнение скрипта
4. Удаление временных отладочных выводов после проверки

Подсказка: Отладка — это итеративный процесс: проверяем, исправляем, проверяем снова.

Ответ:

- Добавление print() для проверки значений
- Исправление ошибок, если результат неправильный
- Повторное выполнение скрипта
- Удаление временных отладочных выводов после проверки

Урок 5 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты только что открыл Lua-скрипт и видишь переменные без явной типизации. Какой типизация используется в Lua?

Подсказка: Подумай, как Lua определяет тип переменной во время выполнения.

Ответ:

- **Динамическая типизация**
- Статическая типизация
- Сильная статическая типизация
- Отсутствие типизации

Шаг 02:

Миссия 2

Ты запускаешь Lua-интерпретатор, чтобы проверить простую функцию. Какая команда открывает интерактивную сессию Lua?

Подсказка: Подумай, что нужно ввести в терминале, чтобы начать писать код построчно.

Ответ:

- **lua**
- runlua
- luainterpreter
- startlua

Шаг 03:

Миссия 3

Ты хочешь создать функцию, которая возвращает сумму двух чисел. Какая запись правильная в Lua?

Подсказка: В Lua функции определяются ключевым словом 'function'.

Ответ:

- **function sum(a, b) return a + b end**
- def sum(a, b): return a + b
- sum(a, b) => a + b
- function sum(a, b) { return a + b }

Шаг 04:

Миссия 4

Формулировка: Сопоставьте конструкции Lua с их назначением.

Понятия:

Ответ:

- if ... then ... end
- for i = 1, 10 do ... end
- function ... end
- local

Определения:

1. Создание локальной переменной
2. Определение функции
3. Цикл с определённым количеством итераций
4. Условный оператор

Подсказка: Подумай о том, какие ключевые слова Lua управляют потоком и областью видимости.

- **if ... then ... end** — Условный оператор
- **for i = 1, 10 do ... end** — Цикл с определённым количеством итераций
- **function ... end** — Определение функции
- **local** — Создание локальной переменной

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы данных Lua с их примером.

Понятия:

Ответ:

- number
- string
- boolean
- table

Определения:

1. 42, 3.14
2. "hello", 'world'
3. true, false
4. {1,2,3}, {name='Alice'}

Подсказка: Подумай, какие структуры хранят числа, текст, логические значения и коллекции.

- **number** — 42, 3.14
- **string** — "hello", 'world'
- **boolean** — true, false
- **table** — {1,2,3}, {name='Alice'}

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте шаги тестирования Lua-скрипта с описанием.

Понятия:

Ответ:

- Запуск скрипта в интерпретаторе
- Проверка синтаксиса
- Отладка и исправление ошибок
- Использование скрипта в приложении

Определения:

1. Запуск кода, чтобы увидеть результат выполнения
2. Поиск ошибок в коде и их исправление
3. Проверка корректности написания команд и функций
4. Интеграция проверенного кода в рабочую систему

Подсказка: Тестирование похоже на квест: сначала проверяешь синтаксис, потом запускаешь, исправляешь и интегрируешь.

- **Запуск скрипта в интерпретаторе** — Запуск кода, чтобы увидеть результат выполнения
- **Проверка синтаксиса** — Проверка корректности написания команд и функций
- **Отладка и исправление ошибок** — Поиск ошибок в коде и их исправление
- **Использование скрипта в приложении** — Интеграция проверенного кода в рабочую систему

Шаг 07:

Миссия 7

Вопрос: Расположите этапы создания простого Lua-скрипта в правильном порядке:

1. Определение цели и логики скрипта
2. Написание кода в текстовом файле или интерпретаторе
3. Тестирование и исправление ошибок
4. Использование скрипта в приложении

Подсказка: Сначала продумываем, потом пишем, тестируем и применяем.

Ответ:

- Определение цели и логики скрипта
- Написание кода в текстовом файле или интерпретаторе
- Тестирование и исправление ошибок
- Использование скрипта в приложении

Шаг 08:

Миссия 8

Вопрос: Расположите шаги запуска Lua-скрипта в интерпретаторе:

1. Открыть терминал
2. Запустить интерпретатор командой 'lua'
3. Ввести код или вызвать файл с кодом
4. Получить результат выполнения и проверить его

Подсказка: Подумай, как проходит интерактивная работа с Lua, шаг за шагом.

Ответ:

- Открыть терминал
- Запустить интерпретатор командой 'lua'
- Ввести код или вызвать файл с кодом
- Получить результат выполнения и проверить его

Шаг 09:

Миссия 9

Вопрос: Расположите действия при отладке Lua-функции:

1. Проверка синтаксиса
2. Запуск функции с тестовыми данными
3. Анализ ошибок и сообщений интерпретатора
4. Исправление логики функции и повторный запуск

Подсказка: Отладка — это цикл проверки и исправления до полного успеха.

Ответ:

- Проверка синтаксиса
- Запуск функции с тестовыми данными
- Анализ ошибок и сообщений интерпретатора
- Исправление логики функции и повторный запуск

Шаг 10:

Миссия 10

Вопрос: Расположите этапы подготовки Lua-скрипта к использованию в Redis:

1. Написание и тестирование скрипта локально
2. Оптимизация кода и проверка производительности
3. Отправка скрипта на сервер Redis
4. Интеграция и мониторинг работы в приложении

Подсказка: Подготовка скрипта — это путь от локального к боевому использованию.

Ответ:

- Написание и тестирование скрипта локально
- Оптимизация кода и проверка производительности
- Отправка скрипта на сервер Redis
- Интеграция и мониторинг работы в приложении

Модуль 3 - Введение в Lua-скрипты в Redis

 Урок 1 - Как работает Lua в Redis sandbox, контекст исполнения

 Урок 2 - Основные команды для запуска скриптов EVAL, EVALSHA

 Урок 3 - Аргументы в скриптах KEYS и ARGV

 Урок 4 - Особенности среды Redis Lua (нет доступа к ОС, только Redis API)

 Урок 5 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты только что узнал, что Lua-скрипты выполняются внутри Redis в sandbox. Что это значит?

Подсказка: Подумай о безопасности и ограничениях при выполнении кода.

Ответ:

- **Скрипт работает в ограниченном окружении, без доступа к ОС**
- Скрипт может запускать любые системные команды
- Скрипт выполняется на клиентской машине
- Скрипт автоматически получает root-доступ к серверу

Шаг 02:

Миссия 2

Ты хочешь выполнить Lua-скрипт в Redis. Какая команда используется для этого напрямую?

Подсказка: Подумай, какая команда позволяет выполнить скрипт передавая его текст.

Ответ:

- **EVAL**
- EVALSHAKE
- EXECUTE
- RUNLUA

Шаг 03:

Миссия 3

Ты сохранил Lua-скрипт в Redis и хочешь вызвать его по хэшу. Какая команда для этого используется?

Подсказка: Вспомни, что Redis может идентифицировать скрипт не по тексту, а по его SHA1.

Ответ:

- **EVALSHA**
- CALLSHA
- RUNSHA
- EXECSHA

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элементы Lua-скрипта в Redis с их ролью.

Понятия:

Ответ:

- KEYS
- ARGV
- EVAL
- EVALSHA

Определения:

1. Список ключей, к которым скрипт обращается
2. Список аргументов, переданных скрипту
3. Выполнение скрипта по тексту
4. Выполнение скрипта по SHA1-хэшу

Подсказка: Подумай, какие конструкции нужны для передачи данных и вызова скрипта.

- **KEYS** — Список ключей, к которым скрипт обращается
- **ARGV** — Список аргументов, переданных скрипту
- **EVAL** — Выполнение скрипта по тексту
- **EVALSHA** — Выполнение скрипта по SHA1-хэшу

Шаг 05:

Миссия 5

Формулировка: Сопоставьте свойства среды Lua в Redis с их объяснением.

Понятия:

Ответ:

- Sandbox
- Доступ к ОС
- Redis API
- Ограничение времени выполнения

Определения:

1. Окружение с ограниченными возможностями для безопасности
2. Возможность использовать только команды Redis

3. Нет доступа к файловой системе или процессам сервера
4. Скрипт не может выполняться бесконечно долго

Подсказка: Подумай о безопасности, API и времени работы скриптов.

- **Sandbox** — Окружение с ограниченными возможностями для безопасности
- **Redis API** — Возможность использовать только команды Redis
- **Доступ к ОС** — Нет доступа к файловой системе или процессам сервера
- **Ограничение времени выполнения** — Скрипт не может выполняться бесконечно долго

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте типы данных, которые Lua-скрипт может получить через KEYS и ARGV, с примером.

Понятия:

Ответ:

- KEYS
- ARGV

Определения:

1. Список ключей базы данных, например {"user:1","user:2"}
2. Аргументы для скрипта, например {"10","active"}

Подсказка: KEYS — это ключи Redis, ARGV — дополнительные параметры для скрипта.

- **KEYS** — Список ключей базы данных, например {"user:1","user:2"}
- **ARGV** — Аргументы для скрипта, например {"10","active"}

🔮 Шаг 07:

Миссия 7

Вопрос: Расположите шаги выполнения Lua-скрипта в Redis:

1. Передача скрипта и аргументов через EVAL
2. Создание списка KEYS и ARGV
3. Выполнение команд внутри sandbox
4. Возврат результата скрипта клиенту

Подсказка: Сначала формируем входные данные, затем отправляем скрипт, выполняем и получаем результат.

Ответ:

- Создание списка KEYS и ARGV
- Передача скрипта и аргументов через EVAL
- Выполнение команд внутри sandbox
- Возврат результата скрипта клиенту

🔮 Шаг 08:

Миссия 8

Вопрос: Расположите этапы вызова сохранённого Lua-скрипта по SHA1-хэшу:

1. Создать скрипт и получить его SHA1

2. Передать SHA1 и аргументы через EVALSHA
3. Выполнить команды внутри Redis
4. Получить результат выполнения на клиенте

Подсказка: SHA1 позволяет идентифицировать скрипт без повторной передачи текста.

Ответ:

- Создать скрипт и получить его SHA1
- Передать SHA1 и аргументы через EVALSHA
- Выполнить команды внутри Redis
- Получить результат выполнения на клиенте

Урок 6 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты только что открыл Redis и решил попробовать Lua-скрипт. Где выполняется Lua-код внутри Redis?

Подсказка: Подумай о безопасной песочнице, где код изолирован от операционной системы.

Ответ:

- **Внутри безопасного sandbox Redis**
- Прямо в операционной системе сервера
- На клиенте, который подключается к Redis
- В отдельном виртуальном окружении, независимом от Redis

Шаг 02:

Миссия 2

Ты хочешь выполнить Lua-скрипт напрямую на сервере Redis. Какой командой это делается?

Подсказка: Подумай о том, как отправить текст скрипта Redis для исполнения.

Ответ:

- **EVAL**
- RUNLUA
- EXECUTE
- SCRIPTLOAD

Шаг 03:

Миссия 3

Ты решил ускорить выполнение скрипта, используя его хэш. Какая команда запускает скрипт по SHA1-хэшу?

Подсказка: Подумай, что Redis использует для кэширования скриптов и быстрого вызова.

Ответ:

- **EVALSHA**
- EVALHASH
- RUNSHA
- SCRIPTEXEC

Шаг 04:

Миссия 4

Формулировка: Сопоставьте параметры Lua-скрипта с их назначением.

Понятия:

Ответ:

- KEYS
- ARGV

Определения:

1. Список ключей Redis, к которым обращается скрипт
2. Список дополнительных аргументов, передаваемых скрипту

Подсказка: KEYS — это ключи, ARGV — дополнительные данные для логики.

- **KEYS** — Список ключей Redis, к которым обращается скрипт
- **ARGV** — Список дополнительных аргументов, передаваемых скрипту

Шаг 05:

Миссия 5

Формулировка: Сопоставьте ограничения среды Lua в Redis с их описанием.

Понятия:

Ответ:

- Нет доступа к ОС
- Доступ только к Redis API
- Sandbox

Определения:

1. Скрипт не может выполнять команды ОС, например читать файлы
2. Можно использовать команды Redis для работы с ключами и значениями
3. Код выполняется в изолированной безопасной среде

Подсказка: Подумай, что защищает сервер и ограничивает возможности скрипта.

- **Нет доступа к ОС** — Скрипт не может выполнять команды ОС, например читать файлы
- **Доступ только к Redis API** — Можно использовать команды Redis для работы с ключами и значениями
- **Sandbox** — Код выполняется в изолированной безопасной среде

Шаг 06:

Миссия 6

Формулировка: Сопоставьте команды Redis с их ролью при работе с Lua-скриптами.

Понятия:

Ответ:

- EVAL
- EVALSHA
- SCRIPT LOAD

Определения:

1. Выполняет Lua-скрипт по тексту
2. Выполняет Lua-скрипт по SHA1-хэшу
3. Загружает скрипт и возвращает его SHA1-хэш

Подсказка: Подумай о разных способах запуска скриптов и кеширования.

- **EVAL** — Выполняет Lua-скрипт по тексту
- **EVALSHA** — Выполняет Lua-скрипт по SHA1-хэшу
- **SCRIPT LOAD** — Загружает скрипт и возвращает его SHA1-хэш

Шаг 07:

Миссия 7

Вопрос: Расположите шаги выполнения Lua-скрипта в Redis в правильном порядке:

1. Написание скрипта
2. Отправка скрипта на сервер с помощью EVAL или EVALSHA
3. Выполнение скрипта в sandbox
4. Возврат результата клиенту

Подсказка: Подумай, как данные проходят путь от написания до результата.

Ответ:

- Написание скрипта
- Отправка скрипта на сервер с помощью EVAL или EVALSHA
- Выполнение скрипта в sandbox
- Возврат результата клиенту

Шаг 08:

Миссия 8

Вопрос: Расположите процесс передачи аргументов в Lua-скрипт Redis:

1. Определение ключей и аргументов в клиенте
2. Передача KEYS и ARGV в команду EVAL
3. Доступ к KEYS и ARGV внутри скрипта
4. Использование данных для логики скрипта

Подсказка: KEYS и ARGV позволяют скрипту работать с внешними данными безопасно.

Ответ:

- Определение ключей и аргументов в клиенте
- Передача KEYS и ARGV в команду EVAL
- Доступ к KEYS и ARGV внутри скрипта
- Использование данных для логики скрипта

Шаг 09:

Миссия 9

Вопрос: Расположите этапы безопасного выполнения Lua-скрипта в Redis:

1. Написание скрипта
2. Проверка скрипта на доступ только к Redis API
3. Запуск скрипта в sandbox

4. Возврат результата клиенту

Подсказка: Подумай, что защищает сервер и пользователей от опасного кода.

Ответ:

- Написание скрипта
- Проверка скрипта на доступ только к Redis API
- Запуск скрипта в sandbox
- Возврат результата клиенту

Модуль 4 - Redis команды в Lua

Урок 1 - Основные команды Redis, доступные в Lua (redis.call, redis.pcall)

Урок 2 - Возврат значений из Lua в Redis

Урок 3 - Обработка ошибок в Lua-скриптах

Урок 4 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты запускаешь свой первый Lua-скрипт в Redis. Какой командой вызывается Redis-функция внутри Lua?

Подсказка: Подумай о том, как Lua обращается к Redis изнутри.

Ответ:

- **redis.call**
- redis.run
- redis.exec
- redis.eval

Шаг 02:

Миссия 2

Ты хочешь выполнить команду Redis внутри Lua, но не хочешь, чтобы ошибка прервала скрипт. Какой метод лучше использовать?

Подсказка: Подумай о безопасном способе вызова, который не ломает выполнение скрипта.

Ответ:

- **redis.pcall**
- redis.call_safe
- redis.try
- redis.eval_safe

Шаг 03:

Миссия 3

Ты возвращаешь результат Lua-скрипта в Redis. Как Lua возвращает значения клиенту?

Подсказка: Подумай о ключевом слове Lua, которое завершает функцию и возвращает результат.

Ответ:

- **return**
- yield
- output
- send

Шаг 04:

Миссия 4

Формулировка: Сопоставьте функции Lua для Redis с их описанием.

Понятия:

Ответ:

- redis.call
- redis.pcall
- return

Определения:

1. Выполняет команду Redis и вызывает ошибку при неудаче
2. Выполняет команду Redis, но возвращает ошибку как значение, не прерывая скрипт
3. Возвращает результат из Lua-скрипта в Redis

Подсказка: Подумай, какие функции вызывают команды, а что возвращает результат.

- **redis.call** — Выполняет команду Redis и вызывает ошибку при неудаче
- **redis.pcall** — Выполняет команду Redis, но возвращает ошибку как значение, не прерывая скрипт
- **return** — Возвращает результат из Lua-скрипта в Redis

Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы ошибок с поведением скрипта при использовании redis.call и redis.pcall.

Понятия:

Ответ:

- Ошибка при redis.call
- Ошибка при redis.pcall

Определения:

1. Скрипт прерывается и возвращает ошибку клиенту
2. Скрипт продолжает выполнение, ошибка возвращается как значение

Подсказка: Подумай, какая команда безопаснее для экспериментов.

- **Ошибка при redis.call** — Скрипт прерывается и возвращает ошибку клиенту
- **Ошибка при redis.pcall** — Скрипт продолжает выполнение, ошибка возвращается как значение

Шаг 06:

Миссия 6

Формулировка: Сопоставьте типы возвращаемых значений из Lua-скрипта с их примерами.

Понятия:

Ответ:

- Строка
- Число
- Массив

Определения:

1. 'OK' или 'Hello'
2. 42 или 3.14
3. {'a', 'b', 'c'}

Подсказка: Подумай о том, какие Lua-тип возвращается клиенту как результат скрипта.

- **Строка** — 'OK' или 'Hello'
- **Число** — 42 или 3.14
- **Массив** — {'a', 'b', 'c'}

Шаг 07:

Миссия 7

Вопрос: Расположите шаги обработки команды Redis в Lua-скрипте:

1. Вызываем команду через redis.call или redis.pcall
2. Обрабатываем результат или ошибку
3. Возвращаем результат клиенту

Подсказка: Подумай о логической последовательности: вызов, проверка, возврат.

Ответ:

- Вызываем команду через redis.call или redis.pcall
- Обрабатываем результат или ошибку
- Возвращаем результат клиенту

Шаг 08:

Миссия 8

Вопрос: Расположите шаги безопасного вызова Redis-команд с обработкой ошибок:

1. Вызов команды через redis.pcall
2. Проверка возвращённого значения на наличие ошибки
3. Продолжение выполнения скрипта при ошибке
4. Возврат успешного или обработанного результата

Подсказка: redis.pcall позволяет скрипту не падать и корректно вернуть данные.

Ответ:

- Вызов команды через redis.pcall
- Проверка возвращённого значения на наличие ошибки
- Продолжение выполнения скрипта при ошибке
- Возврат успешного или обработанного результата

Шаг 09:

Миссия 9

Вопрос: Расположите процесс написания Lua-скрипта с возвратом значения и обработкой ошибок:

1. Определяем ключи и аргументы
2. Выполняем команды через `redis.pcall`
3. Обрабатываем возможные ошибки
4. Возвращаем результат через `return`

Подсказка: Подумай, как правильно встроить безопасное выполнение и возврат данных клиенту.

Ответ:

- Определяем ключи и аргументы
- Выполняем команды через `redis.pcall`
- Обрабатываем возможные ошибки
- Возвращаем результат через `return`

Шаг 10:

Миссия 10

Вопрос: Расположите этапы разработки надежного Lua-скрипта с вызовами Redis:

1. Написание скрипта с логикой и командами Redis
2. Использование `redis.pcall` для безопасных вызовов
3. Обработка ошибок и логирование
4. Тестирование и возврат результата клиенту

Подсказка: Сначала пишем код, потом делаем его безопасным и проверяем перед возвратом данных.

Ответ:

- Написание скрипта с логикой и командами Redis
- Использование `redis.pcall` для безопасных вызовов
- Обработка ошибок и логирование
- Тестирование и возврат результата клиенту

Урок 5 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты начинаешь писать первый Lua-скрипт для Redis. Какая команда используется для вызова Redis-функций внутри Lua?

Подсказка: Подумай, какой метод позволяет Lua взаимодействовать напрямую с Redis.

Ответ:

- **`redis.call`**
- `redis.run`
- `redis.exec`
- `redis.eval`

🔮 Шаг 02:

Миссия 2

Ты хочешь выполнить команду Redis, но не хочешь, чтобы ошибка прервала скрипт. Какой метод безопаснее использовать?

Подсказка: Подумай, какая функция возвращает ошибку как значение, не останавливая скрипт.

Ответ:

- **redis.pcall**
- redis.safe_call
- redis.try
- redis.eval_safe

🔮 Шаг 03:

Миссия 3

Как Lua-скрипт возвращает результат клиенту Redis?

Подсказка: Подумай о ключевом слове Lua, которое завершает функцию и возвращает результат.

Ответ:

- **return**
- yield
- output
- send

🔮 Шаг 04:

Миссия 4

Формулировка: Сопоставьте функции Lua с их назначением при работе с Redis.

Понятия:

Ответ:

- redis.call
- redis.pcall
- return

Определения:

1. Выполняет команду Redis и прерывает скрипт при ошибке
2. Выполняет команду Redis, но возвращает ошибку как значение, не прерывая скрипт
3. Возвращает результат скрипта клиенту

Подсказка: Подумай, какие функции вызывают команды, а что возвращает результат.

- **redis.call** — Выполняет команду Redis и прерывает скрипт при ошибке
- **redis.pcall** — Выполняет команду Redis, но возвращает ошибку как значение, не прерывая скрипт
- **return** — Возвращает результат скрипта клиенту

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте, как ошибки обрабатываются в Lua-скриптах при `redis.call` и `redis.pcall`.

Понятия:

Ответ:

- Ошибка при `redis.call`
- Ошибка при `redis.pcall`

Определения:

1. Скрипт прерывается и возвращает ошибку клиенту
2. Скрипт продолжает выполнение, ошибка возвращается как значение

Подсказка: Подумай, какая команда безопаснее для экспериментов.

- **Ошибка при `redis.call`** — Скрипт прерывается и возвращает ошибку клиенту
- **Ошибка при `redis.pcall`** — Скрипт продолжает выполнение, ошибка возвращается как значение

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте типы возвращаемых значений Lua-скрипта с примерами.

Понятия:

Ответ:

- Строка
- Число
- Массив

Определения:

1. 'OK' или 'Hello'
2. 42 или 3.14
3. {'a', 'b', 'c'}

Подсказка: Подумай о типах Lua, которые возвращаются клиенту.

- **Строка** — 'OK' или 'Hello'
- **Число** — 42 или 3.14
- **Массив** — {'a', 'b', 'c'}

🔮 Шаг 07:

Миссия 7

Вопрос: Расположите шаги обработки Redis-команды в Lua-скрипте:

1. Вызов команды через `redis.call` или `redis.pcall`
2. Обработка результата или ошибки
3. Возврат результата клиенту

Подсказка: Подумай о логической последовательности: сначала вызов, потом проверка, затем возврат.

Ответ:

- Вызов команды через `redis.call` или `redis.pcall`
- Обработка результата или ошибки

- Возврат результата клиенту

🔮 Шаг 08:

Миссия 8

Вопрос: Расположите шаги безопасного вызова Redis-команд с обработкой ошибок:

1. Вызов команды через `redis.pcall`
2. Проверка возвращённого значения на наличие ошибки
3. Продолжение выполнения скрипта при ошибке
4. Возврат успешного или обработанного результата

Подсказка: `redis.pcall` позволяет скрипту не падать и корректно вернуть данные.

Ответ:

- Вызов команды через `redis.pcall`
- Проверка возвращённого значения на наличие ошибки
- Продолжение выполнения скрипта при ошибке
- Возврат успешного или обработанного результата

📖 Модуль 5 - Паттерны использования Lua-скриптов в Redis

📖 Урок 1 - Атомарные инкременты и проверка значений

📖 Урок 2 - Установка TTL вместе с INCR

📖 Урок 3 - Работа с множеством ключей (atomic multi-key operations)

📖 Урок 4 - Использование Lua-скриптов для реализации rate limiting

📖 Урок 5 - Примеры ограничения частоты, счётчики, блокировки

📖 Урок 6 - Квест по мотивам реальной работы программистов — 1

🔮 Шаг 01:

Миссия 1

Ты разработчик и хочешь атомарно увеличить значение счётчика в Redis. Какой подход самый безопасный?

Подсказка: Подумай о том, чтобы операция была атомарной, чтобы два клиента не испортили счётчик.

Ответ:

- **Использовать Lua-скрипт с `redis.call('INCR', key)`**
- Просто сделать `get` и `set` в двух командах
- Использовать внешнюю блокировку клиента
- Использовать `WATCH` без Lua

🔮 Шаг 02:

Миссия 2

Ты хочешь одновременно увеличить значение ключа и установить TTL. Как это можно сделать атомарно в Redis?

Подсказка: Подумай, как объединить несколько команд в один атомарный вызов.

Ответ:

- **Использовать Lua-скрипт с `redis.call('INCR', key)` и `redis.call('EXPIRE', key, ttl)`**
- Сначала INCR, потом EXPIRE отдельно
- Ставить TTL через клиент после INCR
- Использовать SETEX без Lua

🔧 Шаг 03:

Миссия 3

Ты работаешь с несколькими ключами одновременно. Какая стратегия обеспечивает атомарность операций над множеством ключей?

Подсказка: Подумай, как Lua объединяет несколько команд в одну транзакцию.

Ответ:

- **Выполнять все команды внутри одного Lua-скрипта**
- Использовать несколько отдельных команд
- Просто полагаться на клиентскую блокировку
- Использовать WATCH/MULTI без Lua

🔧 Шаг 04:

Миссия 4

Формулировка: Сопоставьте паттерны использования Lua с их примерами.

Понятия:

Ответ:

- Атомарный инкремент
- INCR с TTL
- Множественные ключи
- Rate limiting

Определения:

1. Увеличение значения ключа без риска гонки данных
2. Одновременное увеличение счётчика и установка времени жизни
3. Обработка нескольких ключей в одной операции
4. Ограничение частоты действий одного пользователя

Подсказка: Подумай, где важна атомарность, где TTL, а где контроль частоты.

- **Атомарный инкремент** — Увеличение значения ключа без риска гонки данных
- **INCR с TTL** — Одновременное увеличение счётчика и установка времени жизни
- **Множественные ключи** — Обработка нескольких ключей в одной операции
- **Rate limiting** — Ограничение частоты действий одного пользователя

🔧 Шаг 05:

Миссия 5

Формулировка: Сопоставьте примеры Lua-скриптов с их функцией в Redis.

Понятия:

Ответ:

- Установка TTL после INCR
- Атомарная проверка и инкремент
- Ограничение запросов на пользователя

Определения:

1. `redis.call('INCR', key); redis.call('EXPIRE', key, ttl)`
2. `if redis.call('GET', key) < limit then redis.call('INCR', key)`
3. Скрипт отслеживает количество действий и блокирует превышение лимита

Подсказка: Смотри на комбинации команд и их смысл.

- **Установка TTL после INCR** — `redis.call('INCR', key); redis.call('EXPIRE', key, ttl)`
- **Атомарная проверка и инкремент** — `if redis.call('GET', key) < limit then redis.call('INCR', key)`
- **Ограничение запросов на пользователя** — Скрипт отслеживает количество действий и блокирует превышение лимита

 Шаг 06:

Миссия 6

Вопрос: Расположите шаги создания Lua-скрипта для атомарного инкремента с TTL:

1. Получение текущего значения ключа
2. Проверка лимита (если нужно)
3. Увеличение значения ключа
4. Установка TTL
5. Возврат нового значения клиенту

Подсказка: Сначала проверяем текущее состояние, потом обновляем и устанавливаем TTL.

Ответ:

- Получение текущего значения ключа
- Проверка лимита (если нужно)
- Увеличение значения ключа
- Установка TTL
- Возврат нового значения клиенту

 Шаг 07:

Миссия 7

Вопрос: Расположите шаги для реализации rate limiting через Lua:

1. Проверка текущего счётчика запросов пользователя
2. Сравнение с лимитом
3. **При превышении** — блокировка запроса
4. **Иначе** — увеличение счётчика и возврат ОК

Подсказка: Лимит проверяется до инкремента, иначе можно пропустить превышение.

Ответ:

- Проверка текущего счётчика запросов пользователя
- Сравнение с лимитом
- **При превышении** — блокировка запроса

- **Иначе** — увеличение счётчика и возврат ОК

🧭 Шаг 08:

Миссия 8

Вопрос: Расположите шаги безопасного инкремента нескольких ключей:

1. Начало Lua-скрипта
2. Обход всех ключей с `redis.call('INCR', key)`
3. Обработка ошибок через `redis.pcall` (если нужно)
4. Возврат новых значений клиенту

Подсказка: Сначала начинаем скрипт, затем работаем с ключами, обрабатываем ошибки и возвращаем результат.

Ответ:

- Начало Lua-скрипта
- Обход всех ключей с `redis.call('INCR', key)`
- Обработка ошибок через `redis.pcall` (если нужно)
- Возврат новых значений клиенту

🧭 Шаг 09:

Миссия 9

Вопрос: Расположите шаги для реализации блокировки ресурса через Lua-скрипт:

1. Проверка существования ключа блокировки
2. **Если нет ключа** — установка ключа с TTL
3. **Если ключ есть** — возврат ошибки или сообщения о занятости
4. Возврат успеха при установке блокировки

Подсказка: Блокировка создаётся только если ключ отсутствует, иначе операция отклоняется.

Ответ:

- Проверка существования ключа блокировки
- **Если нет ключа** — установка ключа с TTL
- **Если ключ есть** — возврат ошибки или сообщения о занятости
- Возврат успеха при установке блокировки

📖 Урок 7 - Квест по мотивам реальной работы программистов — 2

🧭 Шаг 01:

Миссия 1

Ты запускаешь свой первый Lua-скрипт для подсчёта кликов на сайте. Какая команда Redis нужна для атомарного увеличения значения ключа?

Подсказка: Подумай о команде, которая увеличивает число на 1 без гонки условий.

Ответ:

- **INCR**
- SET
- GET
- DEL

Шаг 02:

Миссия 2

Ты хочешь установить TTL одновременно с увеличением счётчика. Какую стратегию лучше использовать?

Подсказка: Lua позволяет объединять несколько команд в атомарную операцию.

Ответ:

- **Комбинировать INCR и EXPIRE в Lua-скрипте**
- Сначала INCR, потом EXPIRE отдельными командами
- Использовать только EXPIRE
- INCR с аргументом TTL

Шаг 03:

Миссия 3

Для выполнения атомарной операции с несколькими ключами тебе нужно:

Подсказка: Подумай, как Lua позволяет объединять действия с разными ключами без гонок.

Ответ:

- **Использовать Lua-скрипт, который читает и изменяет все ключи атомарно**
- Выполнять команды по отдельности клиентом
- Создать транзакцию MULTI/EXEC вне Lua
- Использовать WATCH на каждый ключ

Шаг 04:

Миссия 4

Формулировка: Сопоставьте паттерны Lua-скриптов с их назначением.

Понятия:

Ответ:

- Атомарный инкремент
- Установка TTL с INCR
- Работа с множеством ключей
- Rate limiting

Определения:

1. Увеличение значения ключа без риска гонки данных
2. Установка времени жизни ключа вместе с увеличением счётчика
3. Изменение нескольких ключей в одной атомарной операции
4. Ограничение частоты действий пользователя

Подсказка: Подумай, какие паттерны соответствуют счётчикам, таймерам и ограничениям.

- **Атомарный инкремент** — Увеличение значения ключа без риска гонки данных
- **Установка TTL с INCR** — Установка времени жизни ключа вместе с увеличением счётчика
- **Работа с множеством ключей** — Изменение нескольких ключей в одной атомарной операции
- **Rate limiting** — Ограничение частоты действий пользователя

Шаг 05:

Миссия 5

Формулировка: Сопоставьте примеры Lua-скриптов с их использованием в Redis.

Понятия:

Ответ:

- Счётчик посещений
- Блокировка ресурса
- Ограничение запросов
- Атомарное обновление нескольких ключей

Определения:

1. INCR с TTL для каждого пользователя
2. SETNX с TTL
3. INCR и проверка лимита в Lua
4. Чтение и запись нескольких ключей внутри Lua-скрипта

Подсказка: Обрати внимание на цели каждого паттерна — считать, блокировать, ограничивать, обновлять.

- **Счётчик посещений** — INCR с TTL для каждого пользователя
- **Блокировка ресурса** — SETNX с TTL
- **Ограничение запросов** — INCR и проверка лимита в Lua
- **Атомарное обновление нескольких ключей** — Чтение и запись нескольких ключей внутри Lua-скрипта

Шаг 06:

Миссия 6

Формулировка: Сопоставьте ошибки и их обработку в Lua-скриптах для паттернов Redis.

Понятия:

Ответ:

- Ошибка при INCR
- Ошибка при обновлении нескольких ключей

Определения:

1. Скрипт прерывается, но операция атомарна
2. Скрипт не изменяет ни один ключ и возвращает ошибку клиенту

Подсказка: Подумай о разнице между одиночной и множественной атомарной операцией.

- **Ошибка при INCR** — Скрипт прерывается, но операция атомарна
- **Ошибка при обновлении нескольких ключей** — Скрипт не изменяет ни один ключ и возвращает ошибку клиенту

Шаг 07:

Миссия 7

Вопрос: Расположите шаги создания атомарного счётчика с TTL в правильном порядке:

1. Чтение текущего значения ключа
2. Увеличение значения

3. Установка TTL
4. Возврат нового значения клиенту

Подсказка: Сначала определяем значение, потом изменяем, потом задаём TTL, в конце возвращаем результат.

Ответ:

- Чтение текущего значения ключа
- Увеличение значения
- Установка TTL
- Возврат нового значения клиенту

Шаг 08:

Миссия 8

Вопрос: Расположите шаги безопасного обновления нескольких ключей в Lua:

1. Чтение всех ключей
2. Проверка корректности значений
3. Запись новых значений всех ключей
4. Возврат успешного результата

Подсказка: Думай как разработчик, чтобы изменения были атомарными.

Ответ:

- Чтение всех ключей
- Проверка корректности значений
- Запись новых значений всех ключей
- Возврат успешного результата

Шаг 09:

Миссия 9

Вопрос: Расположите шаги реализации rate limiting с Lua-скриптом:

1. Проверка текущего счётчика
2. Сравнение с лимитом
3. Увеличение счётчика
4. Возврат решения клиенту (разрешено/запрещено)

Подсказка: Rate limiting требует сначала проверки, потом действия, и в конце — ответ.

Ответ:

- Проверка текущего счётчика
- Сравнение с лимитом
- Увеличение счётчика
- Возврат решения клиенту (разрешено/запрещено)

Модуль 6 - Практика — реализация rate limiting на Lua

Урок 1 - Разбор Lua-скрипта для fixed window rate limiting

Урок 2 - Реализация token bucket алгоритма в Lua

Урок 3 - Тестирование скриптов с помощью redis-cli

Урок 4 - Встраивание скриптов в приложение на GoPythonNode.js

Урок 5 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты начинаешь писать Lua-скрипт для fixed window rate limiting. Какой подход правильнее для проверки лимита запросов за фиксированный интервал?

Подсказка: Подумай о том, как фиксированное окно хранит счётчик запросов.

Ответ:

- **Использовать счётчик и TTL ключа, обновляемый за фиксированный интервал**
- Хранить список всех времён запросов
- Отправлять все запросы в очередь на сервер
- Игнорировать интервал и считать все запросы за всё время

Шаг 02:

Миссия 2

Ты решаешь реализовать token bucket алгоритм. Что нужно хранить в Redis для каждого пользователя?

Подсказка: Подумай, как token bucket ограничивает количество действий за время.

Ответ:

- **Количество токенов и время последнего обновления**
- Только максимальное количество токенов
- Список всех запросов пользователя
- Только текущий счётчик без времени

Шаг 03:

Миссия 3

Для тестирования скрипта rate limiting через redis-cli, что нужно сделать первым шагом?

Подсказка: Подумай о том, как проверить работу скрипта без приложения.

Ответ:

- **Вызвать Lua-скрипт с ключами и аргументами через EVAL**
- Запустить сервер Redis с опцией --test
- Проверить лог сервера Redis
- Использовать только GET и SET команды

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элементы Lua-скрипта rate limiting с их назначением.

Понятия:

Ответ:

- Счётчик запросов
- TTL ключа
- Максимальное количество токенов
- Время последнего обновления

Определения:

1. Хранит, сколько запросов уже выполнено в текущем окне
2. Определяет окончание текущего окна fixed window
3. Максимальное число действий пользователя в bucket
4. Нужно для расчёта восстановления токенов

Подсказка: Подумай, какая часть фиксирует количество, а какая контролирует время.

- **Счётчик запросов** — Хранит, сколько запросов уже выполнено в текущем окне
- **TTL ключа** — Определяет окончание текущего окна fixed window
- **Максимальное количество токенов** — Максимальное число действий пользователя в bucket
- **Время последнего обновления** — Нужно для расчёта восстановления токенов

Шаг 05:

Миссия 5

Формулировка: Сопоставьте этапы обработки запроса в Lua-скрипте token bucket с их действием.

Понятия:

Ответ:

- Проверка наличия токенов
- Уменьшение токенов
- Вычисление восстановления токенов
- Возврат разрешения/блокировки клиенту

Определения:

1. Скрипт проверяет, достаточно ли токенов для запроса
2. Если токены есть, уменьшаем их количество
3. На основе времени обновляем доступные токены
4. Отправляем результат выполнения клиенту

Подсказка: Подумай о логике алгоритма token bucket шаг за шагом.

- **Проверка наличия токенов** — Скрипт проверяет, достаточно ли токенов для запроса
- **Уменьшение токенов** — Если токены есть, уменьшаем их количество
- **Вычисление восстановления токенов** — На основе времени обновляем доступные токены
- **Возврат разрешения/блокировки клиенту** — Отправляем результат выполнения клиенту

Шаг 06:

Миссия 6

Формулировка: Сопоставьте инструменты и действия при тестировании Lua-скриптов rate limiting.

Понятия:

Ответ:

- redis-cli
- Инструмент для приложения
- Логи сервера

Определения:

1. Позволяет напрямую запускать Lua-скрипт и проверять работу с ключами
2. Используется для интеграции скрипта в Go/Python/Node.js
3. Помогает отследить ошибки и поведение Redis на сервере

Подсказка: Подумай, где удобнее запускать скрипт вручную и где отслеживать ошибки.

- **redis-cli** — Позволяет напрямую запускать Lua-скрипт и проверять работу с ключами
- **Инструмент для приложения** — Используется для интеграции скрипта в Go/Python/Node.js
- **Логи сервера** — Помогает отследить ошибки и поведение Redis на сервере

Шаг 07:

Миссия 7

Вопрос: Расположите шаги выполнения fixed window rate limiting в Lua в правильном порядке:

1. Проверка существующего счётчика
2. Если превышен лимит, вернуть блокировку
3. Увеличение счётчика
4. Установка TTL для ключа, если это первый запрос в окне
5. Возврат разрешения клиенту

Подсказка: Подумай, что сначала проверяется, потом обновляется и только после этого возвращается результат.

Ответ:

- Проверка существующего счётчика
- Если превышен лимит, вернуть блокировку
- Увеличение счётчика
- Установка TTL для ключа, если это первый запрос в окне
- Возврат разрешения клиенту

Шаг 08:

Миссия 8

Вопрос: Расположите шаги token bucket алгоритма в Lua в правильной последовательности:

1. Вычисление количества восстановленных токенов
2. Добавление восстановленных токенов в bucket
3. Проверка доступных токенов для запроса
4. Если есть токены, уменьшение их количества
5. Возврат разрешения или блокировки клиенту

Подсказка: Вспомни, как алгоритм поддерживает постоянное обновление токенов и их расход.

Ответ:

- Вычисление количества восстановленных токенов
- Добавление восстановленных токенов в bucket
- Проверка доступных токенов для запроса
- Если есть токены, уменьшение их количества
- Возврат разрешения или блокировки клиенту

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги интеграции Lua-скрипта rate limiting в приложение на Go/Python/Node.js:

1. Загрузка скрипта в Redis и получение SHA1
2. Вызов скрипта через EVALSHA с ключами и аргументами
3. Обработка результата (разрешено/запрещено)
4. Логирование действий и ошибок

Подсказка: Сначала скрипт нужно загрузить, потом вызывать, обрабатывать результат и логировать.

Ответ:

- Загрузка скрипта в Redis и получение SHA1
- Вызов скрипта через EVALSHA с ключами и аргументами
- Обработка результата (разрешено/запрещено)
- Логирование действий и ошибок

 Шаг 10:

Миссия 10

Вопрос: Расположите последовательность тестирования Lua-скрипта rate limiting через redis-cli:

1. Очистка тестовых ключей в Redis
2. Вызов Lua-скрипта с аргументами для первого запроса
3. Повторные вызовы скрипта, чтобы проверить лимит
4. Проверка TTL и счётчиков ключей
5. Анализ результатов и логов

Подсказка: Сначала подготавливаем среду, потом вызываем скрипт, проверяем лимиты и анализируем результаты.

Ответ:

- Очистка тестовых ключей в Redis
- Вызов Lua-скрипта с аргументами для первого запроса
- Повторные вызовы скрипта, чтобы проверить лимит
- Проверка TTL и счётчиков ключей
- Анализ результатов и логов

 Урок 6 - Квест по мотивам реальной работы программистов — 2

 Шаг 01:

Миссия 1

Ты внедряешь fixed window rate limiting. Какая стратегия определяет, можно ли принять запрос?

Подсказка: Подумай о том, как фиксированные интервалы времени помогают ограничивать частоту.

Ответ:

- **Счётчик запросов в фиксированном окне времени**
- Проверка IP без учёта времени
- Случайный выбор разрешённых запросов
- Использование только TTL без счётчика

Шаг 02:

Миссия 2

Ты хочешь реализовать token bucket алгоритм в Lua. Что хранится в Redis для каждого пользователя?

Подсказка: Подумай, как «ведро» хранит токены для контроля частоты действий.

Ответ:

- **Количество доступных токенов и время последнего обновления**
- Только количество выполненных запросов
- Список IP-адресов
- TTL ключа без счётчика

Шаг 03:

Миссия 3

Ты тестируешь Lua-скрипт с помощью redis-cli. Какой шаг поможет проверить правильность работы rate limiting?

Подсказка: Подумай, как симулировать несколько запросов подряд.

Ответ:

- **Вызов скрипта несколько раз и проверка счётчика**
- Удаление всех ключей после первого запроса
- Только чтение ключей без вызова скрипта
- Вызов скрипта один раз с TTL=0

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элементы rate limiting с их назначением в Lua-скрипте.

Понятия:

Ответ:

- Fixed window
- Token bucket
- TTL
- Счётчик запросов

Определения:

1. Фиксированный интервал времени для ограничения частоты
2. Механизм с токенами для плавного распределения запросов
3. Время жизни ключа в Redis
4. Количество запросов, совершённых пользователем

Подсказка: Подумай, как каждый элемент контролирует нагрузку.

- **Fixed window** — Фиксированный интервал времени для ограничения частоты
- **Token bucket** — Механизм с токенами для плавного распределения запросов
- **TTL** — Время жизни ключа в Redis
- **Счётчик запросов** — Количество запросов, совершённых пользователем

Шаг 05:

Миссия 5

Формулировка: Сопоставьте Redis-команды с их ролью в Lua-скриптах rate limiting.

Понятия:

Ответ:

- INCR
- EXPIRE
- SETNX
- GET

Определения:

1. Увеличивает счётчик запросов
2. Устанавливает TTL для ключа
3. Создаёт ключ, если он не существует (для блокировки)
4. Получает текущее значение ключа

Подсказка: Подумай, какая команда считает, какая контролирует время, а какая защищает от повторного создания.

- **INCR** — Увеличивает счётчик запросов
- **EXPIRE** — Устанавливает TTL для ключа
- **SETNX** — Создаёт ключ, если он не существует (для блокировки)
- **GET** — Получает текущее значение ключа

Шаг 06:

Миссия 6

Формулировка: Сопоставьте этапы тестирования Lua-скриптов rate limiting с действиями в redis-cli.

Понятия:

Ответ:

- Симуляция нескольких запросов
- Проверка счётчика
- Удаление тестовых ключей

Определения:

1. Повторный вызов скрипта через redis-cli
2. GET ключа и проверка значения
3. DEL тестовых ключей после проверки

Подсказка: Думай как QA: сначала тестируем, потом проверяем и очищаем.

- **Симуляция нескольких запросов** — Повторный вызов скрипта через redis-cli
- **Проверка счётчика** — GET ключа и проверка значения

- **Удаление тестовых ключей** — DEL тестовых ключей после проверки

Шаг 07:

Миссия 7

Вопрос: Расположите шаги fixed window rate limiting в правильном порядке:

1. Получение текущего значения счётчика
2. Проверка, не превышен ли лимит
3. Увеличение счётчика
4. Возврат разрешения клиенту

Подсказка: Думай логически: сначала проверяем, потом увеличиваем и сообщаем результат.

Ответ:

- Получение текущего значения счётчика
- Проверка, не превышен ли лимит
- Увеличение счётчика
- Возврат разрешения клиенту

Шаг 08:

Миссия 8

Вопрос: Расположите шаги token bucket алгоритма в Lua:

1. Вычисление доступных токенов с учётом прошедшего времени
2. Проверка наличия хотя бы одного токена
3. Списание токена для запроса
4. Возврат решения клиенту

Подсказка: Сначала обновляем токены, затем проверяем и используем.

Ответ:

- Вычисление доступных токенов с учётом прошедшего времени
- Проверка наличия хотя бы одного токена
- Списание токена для запроса
- Возврат решения клиенту

Шаг 09:

Миссия 9

Вопрос: Расположите шаги тестирования Lua-скрипта rate limiting с redis-cli и очисткой:

1. Симуляция запросов через вызовы скрипта
2. Проверка счётчика и лимита через GET
3. Анализ результатов (разрешено/запрещено)
4. Удаление тестовых ключей

Подсказка: Думай как тестировщик: сначала тест, потом проверка, анализ и чистка.

Ответ:

- Симуляция запросов через вызовы скрипта
- Проверка счётчика и лимита через GET
- Анализ результатов (разрешено/запрещено)
- Удаление тестовых ключей

Шаг 10:

Миссия 10

Вопрос: Расположите шаги встраивания Lua-скрипта rate limiting в приложение на Go/Python/Node.js:

1. Загрузка скрипта в Redis с помощью EVAL или SCRIPT LOAD
2. Вызывание скрипта при каждом запросе клиента
3. Обработка возвращённого значения (разрешено/запрещено)
4. Логирование и мониторинг работы скрипта

Подсказка: Сначала загружаем скрипт, потом используем, проверяем результат и ведём лог.

Ответ:

- Загрузка скрипта в Redis с помощью EVAL или SCRIPT LOAD
- Вызывание скрипта при каждом запросе клиента
- Обработка возвращённого значения (разрешено/запрещено)
- Логирование и мониторинг работы скрипта

Модуль 7 - Оптимизация и безопасность Lua-скриптов

Урок 1 - Оптимизация производительности

Урок 2 - Ограничения времени исполнения скриптов в Redis

Урок 3 - Безопасность sandbox и предотвращение DoS-атак

Урок 4 - Работа с большими скриптами (поддержка EVALSHA)

Урок 5 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты оптимизируешь Lua-скрипт для Redis. Что помогает ускорить выполнение скрипта?

Подсказка: Подумай о том, как уменьшить количество обращений к Redis и повторяющихся операций.

Ответ:

- **Использование локальных переменных и минимизация обращений к Redis**
- Добавление большого количества print для отладки
- Разбивать скрипт на множество мелких скриптов без причины
- Частое использование глобальных переменных

Шаг 02:

Миссия 2

Redis ограничивает время исполнения Lua-скриптов. Какое поведение верно?

Подсказка: Подумай о сценариях долгих циклов в скрипте и последствиях для сервера.

Ответ:

- **Скрипт может быть остановлен Redis при превышении лимита времени**
- Скрипт выполняется бесконечно без ограничений
- Redis автоматически делит скрипт на части для выполнения

- Скрипт получает больше ресурсов, чем другие команды

🔮 Шаг 03:

Миссия 3

Ты работаешь над безопасностью Lua-скриптов. Что помогает предотвратить DoS-атаки?

Подсказка: Подумай о механизмах ограничения ресурсов и sandbox.

Ответ:

- **Использование sandbox и ограничение длительности скрипта**
- Разрешение скриптам доступ ко всем файлам сервера
- Запуск скриптов без проверки входных данных
- Игнорирование лимитов времени и памяти

🔮 Шаг 04:

Миссия 4

Формулировка: Сопоставьте оптимизационные техники с их описанием.

Понятия:

Ответ:

- Локальные переменные
- Минимизация обращений к Redis
- Использование EVALSHA
- Предварительное кеширование

Определения:

1. Хранение данных в локальной переменной вместо повторного вычисления
2. Сокращение количества команд, отправляемых на сервер Redis
3. Вызов уже загруженного скрипта по SHA1 вместо полного EVAL
4. Сохранение промежуточных результатов для повторного использования

Подсказка: Подумай, как каждый метод экономит ресурсы и время выполнения.

- **Локальные переменные** — Хранение данных в локальной переменной вместо повторного вычисления
- **Минимизация обращений к Redis** — Сокращение количества команд, отправляемых на сервер Redis
- **Использование EVALSHA** — Вызов уже загруженного скрипта по SHA1 вместо полного EVAL
- **Предварительное кеширование** — Сохранение промежуточных результатов для повторного использования

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте угрозы безопасности Lua-скриптов с мерами защиты.

Понятия:

Ответ:

- DoS-атака через долгий скрипт
- Непроверенные входные данные
- Доступ к файловой системе

- Повторное выполнение скрипта

Определения:

1. Ограничение времени исполнения и sandbox
2. Валидация данных перед использованием
3. Ограничение прав доступа Lua
4. Использование EVALSHA для предотвращения повторной загрузки

Подсказка: Защита требует комбинации ограничений и безопасных практик.

- **DoS-атака через долгий скрипт** — Ограничение времени исполнения и sandbox
- **Непроверенные входные данные** — Валидация данных перед использованием
- **Доступ к файловой системе** — Ограничение прав доступа Lua
- **Повторное выполнение скрипта** — Использование EVALSHA для предотвращения повторной загрузки

Шаг 06:

Миссия 6

Формулировка: Сопоставьте ограничения Redis для Lua-скриптов с их назначением.

Понятия:

Ответ:

- maxmemory-policy
- lua-time-limit
- sandbox
- Единообразный EVALSHA

Определения:

1. Контролирует, как Redis освобождает память при превышении лимитов
2. Ограничивает максимальное время выполнения скрипта
3. Ограничивает доступ Lua к опасным функциям
4. Позволяет запускать скрипты без повторной передачи кода

Подсказка: Каждое ограничение направлено на защиту или оптимизацию сервера.

- **maxmemory-policy** — Контролирует, как Redis освобождает память при превышении лимитов
- **lua-time-limit** — Ограничивает максимальное время выполнения скрипта
- **sandbox** — Ограничивает доступ Lua к опасным функциям
- **Единообразный EVALSHA** — Позволяет запускать скрипты без повторной передачи кода

Шаг 07:

Миссия 7

Вопрос: Расположите шаги оптимизации Lua-скрипта в правильном порядке:

1. Вынос повторяющихся выражений в локальные переменные
2. Снижение количества команд к Redis
3. Использование EVALSHA для повторного вызова
4. Тестирование и профилирование скрипта

Подсказка: Сначала оптимизируем внутренние операции, затем обращение к серверу и повторное использование, и только потом проверяем.

Ответ:

- Вынос повторяющихся выражений в локальные переменные
- Снижение количества команд к Redis
- Использование EVALSHA для повторного вызова
- Тестирование и профилирование скрипта

 Шаг 08:

Миссия 8

Вопрос: Расположите шаги обеспечения безопасности Lua-скрипта:

1. Включение sandbox
2. Проверка входных данных
3. Ограничение времени выполнения скрипта
4. Использование EVALSHA для повторного вызова

Подсказка: Думай как защитник сервера: сначала изолируем, проверяем данные, ограничиваем время, повторно безопасно используем.

Ответ:

- Включение sandbox
- Проверка входных данных
- Ограничение времени выполнения скрипта
- Использование EVALSHA для повторного вызова

 Шаг 09:

Миссия 9

Вопрос: Расположите шаги работы с большим Lua-скриптом в Redis:

1. Разделение на логические блоки и функции
2. Загрузка скрипта через EVALSHA
3. Оптимизация блоков для минимизации обращений к Redis
4. Тестирование скрипта на производительность и безопасность

Подсказка: Сначала структурируем код, затем оптимизируем, загружаем и проверяем.

Ответ:

- Разделение на логические блоки и функции
- Загрузка скрипта через EVALSHA
- Оптимизация блоков для минимизации обращений к Redis
- Тестирование скрипта на производительность и безопасность

 Шаг 10:

Миссия 10

Вопрос: Расположите шаги интеграции безопасного и оптимизированного Lua-скрипта в приложение:

1. Загрузка скрипта в Redis с помощью EVALSHA
2. Вызов скрипта в приложении
3. Обработка результатов и ошибок
4. Логирование и мониторинг

Подсказка: Сначала готовим скрипт в Redis, затем используем, проверяем результаты и отслеживаем работу.

Ответ:

- Загрузка скрипта в Redis с помощью EVALSHA
- Вызов скрипта в приложении
- Обработка результатов и ошибок
- Логирование и мониторинг

Урок 6 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты только что написал Lua-скрипт в Redis, и он выполняется медленно. Что поможет увеличить производительность?

Подсказка: Подумай о том, как уменьшить количество операций с Redis.

Ответ:

- **Сократить количество команд Redis внутри скрипта**
- Добавлять больше комментариев
- Использовать бесконечные циклы для ускорения
- Игнорировать ошибки выполнения

Шаг 02:

Миссия 2

Ты замечаешь, что скрипты могут виснуть и блокировать Redis. Какой механизм ограничивает их выполнение?

Подсказка: Redis умеет завершать скрипты, если они выполняются слишком долго.

Ответ:

- **Ограничение времени исполнения (script-timeout)**
- Автоматическое увеличение памяти
- Случайное прерывание скрипта
- Использование нескольких Lua-сред без таймаута

Шаг 03:

Миссия 3

Ты хочешь защитить Redis от вредоносных скриптов и DoS-атак. Что поможет ограничить возможности Lua?

Подсказка: Подумай о безопасной среде исполнения.

Ответ:

- **Выполнять скрипты в sandbox без доступа к файловой системе и опасным функциям**
- Разрешить выполнение любых Lua-функций
- Игнорировать ошибки и продолжать выполнение
- Ограничить пользователей только по IP

Шаг 04:

Миссия 4

Формулировка: Сопоставьте термины оптимизации Lua-скриптов с их назначением.

Ответ:

- Pipeline
- Меньше циклов
- Кэширование результатов
- Использование EVALSHA

1. Объединение нескольких команд в одну транзакцию для уменьшения RTT
2. Сокращение повторных вычислений в Lua
3. Хранение промежуточных данных для повторного использования
4. Использование хеша скрипта вместо передачи полного текста

Подсказка: Подумай, что сокращает задержки, вычисления и трафик.

- **Pipeline** — Объединение нескольких команд в одну транзакцию для уменьшения RTT
- **Меньше циклов** — Сокращение повторных вычислений в Lua
- **Кэширование результатов** — Хранение промежуточных данных для повторного использования
- **Использование EVALSHA** — Использование хеша скрипта вместо передачи полного текста

Шаг 05:

Миссия 5

Формулировка: Сопоставьте элементы безопасности Lua-скриптов с их функцией.

Ответ:

- Sandbox
- script-timeout
- LIMITED_ENV
- Проверка входных данных

1. Изоляция скрипта от критических функций
2. Ограничение времени исполнения
3. Ограничение доступных глобальных переменных и функций
4. Защита от неожиданных и вредоносных входных данных

Подсказка: Думай о том, что защищает Redis и Lua от злонамеренных действий.

- **Sandbox** — Изоляция скрипта от критических функций
- **script-timeout** — Ограничение времени исполнения
- **LIMITED_ENV** — Ограничение доступных глобальных переменных и функций
- **Проверка входных данных** — Защита от неожиданных и вредоносных входных данных

Шаг 06:

Миссия 6

Формулировка: Сопоставьте преимущества использования EVALSHA для больших скриптов.

Ответ:

- Сокращение сетевого трафика
- Повышение безопасности
- Быстрое повторное выполнение

- Снижение нагрузки на Redis

1. Не передаётся полный текст скрипта при каждом вызове
2. Снижается риск выполнения подменённых скриптов
3. Redis использует уже загруженный скрипт по хешу
4. Меньше вычислительных затрат на парсинг скрипта

Подсказка: Думай о том, что экономит сеть, время и ресурсы.

- **Сокращение сетевого трафика** — Не передаётся полный текст скрипта при каждом вызове
- **Повышение безопасности** — Снижается риск выполнения подменённых скриптов
- **Быстрое повторное выполнение** — Redis использует уже загруженный скрипт по хешу
- **Снижение нагрузки на Redis** — Меньше вычислительных затрат на парсинг скрипта

Шаг 07:

Миссия 7

Вопрос: Расположите шаги оптимизации Lua-скрипта по производительности:

1. Сокращение количества команд Redis
2. Использование EVALSHA
3. Кэширование промежуточных данных
4. Минимизация циклов и вычислений

Подсказка: Думай логически: сначала сокращаем операции, потом ускоряем повторные вызовы и вычисления.

Ответ:

- Сокращение количества команд Redis
- Использование EVALSHA
- Кэширование промежуточных данных
- Минимизация циклов и вычислений

Шаг 08:

Миссия 8

Вопрос: Расположите шаги обеспечения безопасности Lua-скрипта:

1. Выполнение скрипта в sandbox
2. Установка script-timeout
3. Ограничение глобальных переменных (LIMITED_ENV)
4. Проверка входных данных

Подсказка: Думай как защитник: изолируем, ограничиваем, проверяем.

Ответ:

- Выполнение скрипта в sandbox
- Установка script-timeout
- Ограничение глобальных переменных (LIMITED_ENV)
- Проверка входных данных

Шаг 09:

Миссия 9

Вопрос: Расположите шаги работы с большими скриптами в Redis:

1. Загрузка скрипта через EVAL
2. Получение хеша скрипта
3. Вызов скрипта через EVALSHA
4. Повторное выполнение без передачи полного текста

Подсказка: Сначала скрипт загружается, затем используется через хеш.

Ответ:

- Загрузка скрипта через EVAL
- Получение хеша скрипта
- Вызов скрипта через EVALSHA
- Повторное выполнение без передачи полного текста

 **Шаг 10:**

Миссия 10

Вопрос: Расположите последовательность действий при оптимизации и защите Lua-скрипта в проекте:

1. Написание скрипта
2. Тестирование и измерение времени исполнения
3. Оптимизация команд и циклов
4. Применение sandbox и script-timeout
5. Использование EVALSHA для повторного вызова

Подсказка: Сначала пишем и тестируем, затем оптимизируем и защищаем, и в конце ускоряем повторные вызовы.

Ответ:

- Написание скрипта
- Тестирование и измерение времени исполнения
- Оптимизация команд и циклов
- Применение sandbox и script-timeout
- Использование EVALSHA для повторного вызова

Модуль 8 - Продвинутые темы и интеграции

 **Урок 1 - Использование Redis Streams с Lua**

 **Урок 2 - Транзакции и Lua-скрипты**

 **Урок 3 - Мониторинг и логирование Lua-скриптов в Redis**

 **Урок 4 - Кейсы из реальной жизни**

 **Урок 5 - Квест по мотивам реальной работы программистов — 1**

 **Шаг 01:**

Миссия 1

Ты создаёшь поток данных в Redis с помощью Lua. Какой тип структуры лучше использовать для упорядоченных событий?

Подсказка: Подумай о структуре, которая поддерживает append и чтение по порядку.

Ответ:

- **Redis Streams**
- Hash
- Set
- List с push и pop

Шаг 02:

Миссия 2

Ты пишешь Lua-скрипт для обработки транзакции. Что нужно учитывать для атомарности операций?

Подсказка: Lua-скрипт в Redis выполняется как единая атомарная операция.

Ответ:

- **Все команды внутри скрипта выполняются атомарно**
- Каждая команда выполняется независимо, нужно вручную блокировать ключи
- Использовать MULTI/EXEC внутри Lua
- **Ничего не нужно делать** — Redis сам разделяет транзакции

Шаг 03:

Миссия 3

Ты хочешь логировать все ключевые действия Lua-скриптов. Какой подход будет правильным?

Подсказка: Подумай, что можно сохранять внутри Redis для мониторинга событий.

Ответ:

- **Использовать отдельный Stream или List для логирования событий**
- Выводить логи на stdout только
- Создавать отдельные файлы на диске из Lua
- **Не логировать** — Redis сам хранит историю команд

Шаг 04:

Миссия 4

Формулировка: Сопоставьте функции Lua-скриптов в Redis с их практическим применением.

Ответ:

- LPUSH/ RPUSH
 - XADD
 - WATCH
 - INCRBY
1. Добавление элементов в список
 2. Добавление записи в Stream
 3. Контроль изменений ключей перед транзакцией
 4. Увеличение числового поля в хеше

Подсказка: Подумай, где и какие структуры данных применяются.

- **LPUSH/ RPUSH** — Добавление элементов в список
- **XADD** — Добавление записи в Stream
- **WATCH** — Контроль изменений ключей перед транзакцией

- **HINCRBY** — Увеличение числового поля в хеше

Шаг 05:

Миссия 5

Формулировка: Сопоставьте элементы мониторинга Lua-скриптов с их назначением.

Ответ:

- MONITOR
- SLOWLOG
- Логирование через Stream
- Встроенные метрики Redis

1. Печатает все команды, выполняемые сервером
2. Хранит информацию о медленных запросах
3. Позволяет хранить события Lua в потоках для анализа
4. Предоставляет статистику по операциям и ключам

Подсказка: Подумай, что используется для дебага, а что для аналитики.

- **MONITOR** — Печатает все команды, выполняемые сервером
- **SLOWLOG** — Хранит информацию о медленных запросах
- **Логирование через Stream** — Позволяет хранить события Lua в потоках для анализа
- **Встроенные метрики Redis** — Предоставляет статистику по операциям и ключам

Шаг 06:

Миссия 6

Формулировка: Сопоставьте кейсы из реальной жизни с использованием Lua-скриптов в Redis.

Ответ:

- Реализация rate limiting
- Очередь задач
- Атомарное обновление баланса
- Логирование событий пользователя

1. Ограничение частоты запросов API
2. Обработка фоновых задач
3. Снятие/пополнение средств на счете без конфликтов
4. Сбор информации о действиях пользователей

Подсказка: Подумай, какие действия требуют атомарности и упорядочивания.

- **Реализация rate limiting** — Ограничение частоты запросов API
- **Очередь задач** — Обработка фоновых задач
- **Атомарное обновление баланса** — Снятие/пополнение средств на счете без конфликтов
- **Логирование событий пользователя** — Сбор информации о действиях пользователей

Шаг 07:

Миссия 7

Вопрос: Расположите шаги для безопасного использования Lua-скрипта с транзакцией:

1. WATCH нужные ключи
2. Запуск Lua-скрипта

3. MULTI

4. EXEC

Подсказка: Сначала нужно следить за ключами, затем оборачивать в транзакцию.

Ответ:

- WATCH нужные ключи
- MULTI
- Запуск Lua-скрипта
- EXEC

Шаг 08:

Миссия 8

Вопрос: Расположите шаги мониторинга Lua-скриптов в правильном порядке:

1. Включение MONITOR для всех команд
2. Фильтрация команд Lua-скриптов
3. Сохранение важных событий в Stream
4. Анализ данных и метрик

Подсказка: Сначала отслеживаем команды, затем сохраняем и анализируем.

Ответ:

- Включение MONITOR для всех команд
- Фильтрация команд Lua-скриптов
- Сохранение важных событий в Stream
- Анализ данных и метрик

Шаг 09:

Миссия 9

Вопрос: Расположите действия при интеграции Lua-скрипта с Redis Streams:

1. Создание Stream
2. XADD для добавления событий
3. Обработка событий Lua-скриптом
4. ACK подтверждение обработки

Подсказка: Подумай о жизненном цикле события в стриме.

Ответ:

- Создание Stream
- XADD для добавления событий
- Обработка событий Lua-скриптом
- ACK подтверждение обработки

Шаг 10:

Миссия 10

Вопрос: Расположите последовательность действий при реализации кейса из реальной жизни (например, rate limiting и логирование):

1. Получение события от пользователя
2. Проверка лимита через Lua-скрипт

3. Запись события в Stream для логирования
4. Возврат результата пользователю

Подсказка: Сначала получаем данные, затем проверяем и сохраняем, в конце — результат пользователю.

Ответ:

- Получение события от пользователя
- Проверка лимита через Lua-скрипт
- Запись события в Stream для логирования
- Возврат результата пользователю

Урок 6 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты хочешь обрабатывать события в реальном времени с помощью Redis Streams и Lua. Какой подход лучше всего использовать?

Подсказка: Подумай, что позволяет скрипту работать с очередью событий прямо внутри Redis.

Ответ:

- **Использовать Lua-скрипт для обработки сообщений из Streams**
- Сохранять события только в внешней базе
- Игнорировать сообщения и полагаться на периодический polling
- Обрабатывать события вручную через клиент Redis

Шаг 02:

Миссия 2

Ты собираешься выполнять несколько команд Redis атомарно. Что гарантирует целостность данных при использовании Lua?

Подсказка: Подумай, как Redis обрабатывает скрипты.

Ответ:

- **Lua-скрипт выполняется атомарно, гарантируя транзакционность**
- Каждая команда выполняется отдельно и может прерываться
- Нужна дополнительная блокировка ключей вручную
- Только MULTI/EXEC могут гарантировать атомарность

Шаг 03:

Миссия 3

Ты хочешь отслеживать, как Lua-скрипт влияет на Redis. Что поможет получить информацию о его выполнении?

Подсказка: Подумай о встроенных инструментах мониторинга Redis.

Ответ:

- **Использовать MONITOR и логирование внутри скрипта**
- Ставить print в клиентском коде и ждать результатов
- Запускать скрипт без мониторинга
- Сохранять только результат выполнения в файл

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элементы работы с Redis Streams и Lua с их функцией.

Ответ:

- XADD
- XREAD
- XGROUP
- XRANGE

1. Добавление нового события в поток
2. Чтение событий из потока
3. Создание группы потребителей для распределенной обработки
4. Получение диапазона сообщений по ключу и времени

Подсказка: Думай о типичных операциях с потоками: запись, чтение, группы, диапазоны.

- **XADD** — Добавление нового события в поток
- **XREAD** — Чтение событий из потока
- **XGROUP** — Создание группы потребителей для распределенной обработки
- **XRANGE** — Получение диапазона сообщений по ключу и времени

Шаг 05:

Миссия 5

Формулировка: Сопоставьте элементы транзакций Lua с их преимуществами.

Ответ:

- Атомарность
- Изоляция
- Согласованность
- Повторяемость

1. Все команды выполняются как единое целое
2. Скрипт не видит промежуточные состояния других скриптов
3. Данные остаются корректными после выполнения
4. Результат можно повторно вычислить без побочных эффектов

Подсказка: Подумай о принципах ACID в контексте Lua и Redis.

- **Атомарность** — Все команды выполняются как единое целое
- **Изоляция** — Скрипт не видит промежуточные состояния других скриптов
- **Согласованность** — Данные остаются корректными после выполнения
- **Повторяемость** — Результат можно повторно вычислить без побочных эффектов

Шаг 06:

Миссия 6

Формулировка: Сопоставьте подходы к мониторингу Lua-скриптов с их назначением.

Ответ:

- MONITOR
- SLOWLOG
- Логирование ошибок

- Клиентская аналитика

1. Показ всех команд в реальном времени
2. Выявление медленных скриптов
3. Отслеживание неожиданных сбоев в скриптах
4. Сбор метрик и статистики работы скрипта на уровне приложения

Подсказка: Думай о средствах диагностики и анализа работы Lua.

- **MONITOR** — Показ всех команд в реальном времени
- **SLOWLOG** — Выявление медленных скриптов
- **Логирование ошибок** — Отслеживание неожиданных сбоев в скриптах
- **Клиентская аналитика** — Сбор метрик и статистики работы скрипта на уровне приложения

Шаг 07:

Миссия 7

Вопрос: Расположите шаги обработки события в Redis Stream с Lua:

1. XREAD для получения нового события
2. Обработка события внутри Lua-скрипта
3. XACK для подтверждения обработки
4. При необходимости запись результатов через XADD

Подсказка: Думай, что сначала читаем, потом обрабатываем, подтверждаем и сохраняем результаты.

Ответ:

- XREAD для получения нового события
- Обработка события внутри Lua-скрипта
- XACK для подтверждения обработки
- При необходимости запись результатов через XADD

Шаг 08:

Миссия 8

Вопрос: Расположите шаги безопасного выполнения Lua-скрипта с транзакциями:

1. Проверка входных данных
2. Выполнение скрипта внутри Redis
3. Контроль времени выполнения (script-timeout)
4. Логирование результата

Подсказка: Сначала готовим данные, потом выполняем, контролируем время и фиксируем результат.

Ответ:

- Проверка входных данных
- Выполнение скрипта внутри Redis
- Контроль времени выполнения (script-timeout)
- Логирование результата

Шаг 09:

Миссия 9

Вопрос: Расположите этапы интеграции Lua-скриптов в реальную систему:

1. Разработка Lua-скрипта
2. Тестирование в изолированной среде
3. Оптимизация и защита (sandbox, таймаут)
4. Встраивание в приложение (Go/Python/Node.js)
5. Мониторинг и логирование

Подсказка: Думай как полный цикл: пишем, тестируем, защищаем, подключаем и наблюдаем.

Ответ:

- Разработка Lua-скрипта
- Тестирование в изолированной среде
- Оптимизация и защита (sandbox, таймаут)
- Встраивание в приложение (Go/Python/Node.js)
- Мониторинг и логирование

Шаг 10:

Миссия 10

Вопрос: Расположите последовательность действий при работе с кейсом из реальной жизни, используя Redis и Lua:

1. Сбор данных событий и логов
2. Обработка событий через Lua-скрипт
3. Сохранение результатов в Redis
4. Мониторинг и анализ выполнения скриптов
5. Оптимизация и улучшение скриптов на основе данных

Подсказка: Сначала собираем данные, потом обрабатываем, сохраняем, мониторим и улучшаем.

Ответ:

- Сбор данных событий и логов
- Обработка событий через Lua-скрипт
- Сохранение результатов в Redis
- Мониторинг и анализ выполнения скриптов
- Оптимизация и улучшение скриптов на основе данных

Модуль 9 - Итоговый проект

Урок 1 - Разработать собственный Lua-скрипт для Redis

Урок 2 - Интегрировать в выбранное приложение

Урок 3 - Написать тесты и документацию

Урок 4 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты — разработчик, который создаёт свой первый Lua-скрипт для Redis. С чего лучше начать?

Подсказка: Подумай, как обычно начинают создание функционала: с маленького рабочего прототипа.

Ответ:

- **Сначала написать минимальный работающий скрипт с базовой функциональностью**
- Сразу писать весь большой скрипт со всеми фичами
- Только планировать и не писать код
- Воспользоваться готовым скриптом без изменений

Шаг 02:

Миссия 2

Ты разработал Lua-скрипт. Как лучше интегрировать его в приложение (Go/Python/Node.js)?

Подсказка: Подумай о безопасном и удобном способе вызова скрипта из кода приложения.

Ответ:

- **Использовать EVAL/EVALSHA из клиентской библиотеки Redis**
- Копировать код скрипта в приложение и запускать как обычный Lua
- Запускать скрипт вручную через redis-cli каждый раз
- Игнорировать интеграцию и работать только с данными приложения

Шаг 03:

Миссия 3

Ты хочешь убедиться, что скрипт не создаст проблем в продакшене. Что поможет защитить Redis?

Подсказка: Думай о ограничениях на выполнение и безопасности.

Ответ:

- **Установить лимит времени выполнения и использовать sandbox**
- Просто доверять, что скрипт безопасен
- Разрешить скрипту запускать любые системные команды
- Игнорировать возможные ошибки и продолжать работу

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элемент разработки Lua-скрипта с его назначением.

Ответ:

- Минимальный прототип
 - Интеграция с приложением
 - Тестирование
 - Документация
1. Создать рабочий пример с базовым функционалом
 2. Вызов скрипта через EVAL/EVALSHA из приложения
 3. Проверка корректности работы и обработка ошибок
 4. Описание работы скрипта и инструкции для разработчиков

Подсказка: Думай о логике полного цикла разработки: сначала прототип, потом интеграция, тесты, документация.

- **Минимальный прототип** — Создать рабочий пример с базовым функционалом
- **Интеграция с приложением** — Вызов скрипта через EVAL/EVALSHA из приложения

- **Тестирование** — Проверка корректности работы и обработка ошибок
- **Документация** — Описание работы скрипта и инструкции для разработчиков

Шаг 05:

Миссия 5

Формулировка: Сопоставьте типы тестов с их целью.

Ответ:

- Unit-тест
 - Интеграционный тест
 - Нагрузочный тест
 - Документирующий тест
1. Проверяет отдельные функции скрипта
 2. Проверяет взаимодействие Lua-скрипта с приложением и Redis
 3. Проверяет стабильность под высокой нагрузкой
 4. Пример использования и инструкции для разработчиков

Подсказка: Подумай о целях тестирования: от маленьких модулей до полной системы и документации.

- **Unit-тест** — Проверяет отдельные функции скрипта
- **Интеграционный тест** — Проверяет взаимодействие Lua-скрипта с приложением и Redis
- **Нагрузочный тест** — Проверяет стабильность под высокой нагрузкой
- **Документирующий тест** — Пример использования и инструкции для разработчиков

Шаг 06:

Миссия 6

Вопрос: Расположите шаги интеграции Lua-скрипта в приложение.

1. Подключение клиента Redis
2. Вызов скрипта через EVAL/EVALSHA
3. Обработка результата в приложении
4. Обработка ошибок и логирование

Подсказка: Сначала готовим клиент, затем вызываем скрипт, потом обрабатываем результат и ошибки.

Ответ:

- Подключение клиента Redis
- Вызов скрипта через EVAL/EVALSHA
- Обработка результата в приложении
- Обработка ошибок и логирование

Урок 5 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты решил создать свой собственный Lua-скрипт для Redis, чтобы ускорить обработку заказов. С чего стоит начать?

Подсказка: Подумай о базовой структуре скрипта и ключевых командах Redis, которые понадобятся.

Ответ:

- **Определить ключи и действия, которые скрипт будет выполнять**
- Сразу интегрировать скрипт в приложение без тестов
- Начать писать документацию без кода
- Игнорировать обработку ошибок на первом этапе

Шаг 02:

Миссия 2

Ты хочешь, чтобы твой Lua-скрипт работал только с конкретными ключами, чтобы избежать конфликтов. Как это лучше сделать?

Подсказка: Подумай о структуре KEYS и ARGV в Lua-скриптах Redis.

Ответ:

- **Использовать KEYS для указания всех ключей, с которыми будет работать скрипт**
- Передавать ключи как глобальные переменные внутри скрипта
- Жёстко прописывать имена ключей в коде
- Игнорировать ключи, скрипт обработает всё подряд

Шаг 03:

Миссия 3

Перед интеграцией Lua-скрипта в приложение важно проверить его работу. Какой метод тестирования наиболее эффективен на этом этапе?

Подсказка: Подумай о средстве, которое позволяет запускать скрипт напрямую и видеть результат сразу.

Ответ:

- **Тестировать скрипт через redis-cli с разными входными данными**
- Сразу подключать скрипт к продакшн-приложению
- Игнорировать тесты и писать документацию
- Проверять работу скрипта только вручную в приложении

Шаг 04:

Миссия 4

Формулировка: Сопоставьте элементы написания Lua-скрипта с их назначением.

Ответ:

- KEYS
 - ARGV
 - redis.call()
 - return
1. Передача списка ключей, с которыми скрипт будет работать
 2. Передача дополнительных аргументов в скрипт
 3. Выполнение команды Redis внутри скрипта
 4. Возврат результата клиенту

Подсказка: Думай о базовой структуре Lua-скрипта и способах взаимодействия с Redis.

- **KEYS** — Передача списка ключей, с которыми скрипт будет работать
- **ARGV** — Передача дополнительных аргументов в скрипт

- **redis.call()** — Выполнение команды Redis внутри скрипта
- **return** — Возврат результата клиенту

🔗 Шаг 05:

Миссия 5

Формулировка: Сопоставьте этапы интеграции Lua-скрипта в приложение с их смыслом.

Ответ:

- Подключение скрипта к приложению
 - Обработка входящих данных
 - Вызов скрипта через клиент
 - Обработка результата скрипта
1. Приложение умеет отправлять команды Lua в Redis
 2. Подготовка данных, которые скрипт будет использовать
 3. Выполнение скрипта и получение результата
 4. Использование результата в логике приложения

Подсказка: Думай о последовательности действий: подключаем, подготавливаем, вызываем, обрабатываем.

- **Подключение скрипта к приложению** — Приложение умеет отправлять команды Lua в Redis
- **Обработка входящих данных** — Подготовка данных, которые скрипт будет использовать
- **Вызов скрипта через клиент** — Выполнение скрипта и получение результата
- **Обработка результата скрипта** — Использование результата в логике приложения

🔗 Шаг 06:

Миссия 6

Формулировка: Сопоставьте элементы написания тестов и документации с их целью.

Ответ:

- Unit-тесты
 - Интеграционные тесты
 - Документация к скрипту
 - Примеры использования
1. Проверка отдельных функций скрипта
 2. Проверка работы скрипта в связке с приложением
 3. Объяснение, как работает скрипт
 4. Показ реальных сценариев применения скрипта

Подсказка: Думай о тестах как о проверке работы кода, а документации — как о инструкции для пользователя.

- **Unit-тесты** — Проверка отдельных функций скрипта
- **Интеграционные тесты** — Проверка работы скрипта в связке с приложением
- **Документация к скрипту** — Объяснение, как работает скрипт
- **Примеры использования** — Показ реальных сценариев применения скрипта

Модуль 10 - Бонус. Наиболее часто встречающиеся Lua-скрипты для Redis

 Урок 1 - Rate Limiting (ограничение запросов)

 Урок 2 - Sliding Window

 Урок 3 - Distributed Lock

 Урок 4 - Счетчики с TTL

 Урок 5 - Очереди (Queues)

 Урок 6 - Сессии и TTL

 Урок 7 - Атомарные обновления (Compare-And-Set)

 Урок 8 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты хочешь ограничить количество запросов к своему сервису. Какой Lua-скрипт Redis лучше использовать?

Подсказка: Подумай о механизме, который предотвращает слишком частые вызовы.

Ответ:

- **Rate Limiting**
- Distributed Lock
- Счетчики с TTL
- Очереди (Queues)

Шаг 02:

Миссия 2

Тебе нужно хранить историю запросов для анализа за скользящее окно времени. Какой механизм Lua-скрипта выбрать?

Подсказка: Подумай о динамическом окне времени, которое перемещается вперед по мере новых событий.

Ответ:

- **Sliding Window**
- Rate Limiting
- Distributed Lock
- Сессии и TTL

Шаг 03:

Миссия 3

Ты хочешь обеспечить эксклюзивный доступ к ресурсу для одного клиента. Какой Lua-скрипт Redis поможет?

Подсказка: Подумай о механизме, который не позволяет двум процессам одновременно менять данные.

Ответ:

- **Distributed Lock**
- Rate Limiting
- Sliding Window
- Очереди (Queues)

 **Шаг 04:**

Миссия 4

Формулировка: Сопоставьте Lua-скрипт с его назначением.

Ответ:

- Rate Limiting
 - Sliding Window
 - Distributed Lock
 - Счетчики с TTL
 - Очереди (Queues)
 - Сессии и TTL
 - Атомарные обновления (Compare-And-Set)
1. Ограничение количества запросов за период
 2. Хранение событий за скользящее окно времени
 3. Обеспечение эксклюзивного доступа к ресурсу
 4. Подсчет с автоматическим истечением срока жизни
 5. Обработка последовательных задач в порядке поступления
 6. Хранение информации о пользователе с автоматическим удалением
 7. Изменение значения только при совпадении текущего состояния

Подсказка: Подумай о ключевых сценариях использования каждого скрипта в реальной системе.

- **Rate Limiting** — Ограничение количества запросов за период
- **Sliding Window** — Хранение событий за скользящее окно времени
- **Distributed Lock** — Обеспечение эксклюзивного доступа к ресурсу
- **Счетчики с TTL** — Подсчет с автоматическим истечением срока жизни
- **Очереди (Queues)** — Обработка последовательных задач в порядке поступления
- **Сессии и TTL** — Хранение информации о пользователе с автоматическим удалением
- **Атомарные обновления (Compare-And-Set)** — Изменение значения только при совпадении текущего состояния

 **Шаг 05:**

Миссия 5

Формулировка: Сопоставьте сценарий использования скрипта с примером.

Ответ:

- Rate Limiting
- Sliding Window
- Distributed Lock
- Счетчики с TTL
- Очереди (Queues)
- Сессии и TTL

- Атомарные обновления (Compare-And-Set)

1. Не дать пользователю отправлять больше 10 запросов в секунду
2. Отслеживать события за последние 5 минут для аналитики
3. Запретить двум процессам одновременно менять баланс счета
4. Посчитать количество просмотров товара и автоматически сбросить через сутки
5. Обрабатывать платежи в порядке очереди
6. Хранить токен сессии, который истекает через 30 минут
7. Обновить данные профиля только если предыдущая версия совпадает с текущей

Подсказка: Подумай о реальных задачах, которые решают эти скрипты.

- **Rate Limiting** — Не дать пользователю отправлять больше 10 запросов в секунду
- **Sliding Window** — Отслеживать события за последние 5 минут для аналитики
- **Distributed Lock** — Запретить двум процессам одновременно менять баланс счета
- **Счетчики с TTL** — Посчитать количество просмотров товара и автоматически сбросить через сутки
- **Очереди (Queues)** — Обрабатывать платежи в порядке очереди
- **Сессии и TTL** — Хранить токен сессии, который истекает через 30 минут
- **Атомарные обновления (Compare-And-Set)** — Обновить данные профиля только если предыдущая версия совпадает с текущей

Шаг 06:

Миссия 7

Вопрос: Расположите шаги внедрения Lua-скрипта Rate Limiting в продакшн в правильном порядке:

1. Разработка скрипта
2. Тестирование скрипта с разными сценариями
3. Интеграция с приложением
4. Мониторинг работы и логирование
5. Оптимизация производительности при высокой нагрузке

Подсказка: Думай о логической последовательности от написания к эксплуатации.

Ответ:

- Разработка скрипта
- Тестирование скрипта с разными сценариями
- Интеграция с приложением
- Мониторинг работы и логирование
- Оптимизация производительности при высокой нагрузке

Шаг 07:

Миссия 8

Вопрос: Расположите шаги для реализации атомарного обновления (Compare-And-Set) в Lua-скрипте Redis:

1. Получить текущее значение ключа
2. Сравнить с ожидаемым значением
3. Если совпадает, записать новое значение
4. Вернуть результат успеха или отказа клиенту

Подсказка: Подумай о логике проверки перед записью, чтобы избежать гонок.

Ответ:

- Получить текущее значение ключа
- Сравнить с ожидаемым значением
- Если совпадает, записать новое значение
- Вернуть результат успеха или отказа клиенту

Урок 9 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты хочешь ограничить количество запросов к API в Redis. Какой Lua-скрипт лучше всего использовать?

Подсказка: Подумай о контроле частоты действий пользователей.

Ответ:

- **Rate Limiting**
- Sliding Window
- Distributed Lock
- Queues

Шаг 02:

Миссия 2

Ты хочешь динамически контролировать запросы за скользящее время. Какой скрипт Redis подойдёт?

Подсказка: Подумай о концепции "окна времени" для подсчёта событий.

Ответ:

- **Sliding Window**
- Rate Limiting
- Compare-And-Set
- Queues

Шаг 03:

Миссия 3

Чтобы обеспечить атомарный доступ к ресурсу в распределённой системе, какой Lua-скрипт Redis лучше применить?

Подсказка: Думай о блокировке и предотвращении конфликтов.

Ответ:

- **Distributed Lock**
- Rate Limiting
- Queues
- Sliding Window

Шаг 04:

Миссия 4

Формулировка: Сопоставьте тип Lua-скрипта с его описанием.

Ответ:

- Rate Limiting
- Sliding Window
- Distributed Lock
- Compare-And-Set
- Queues

1. Ограничение количества запросов за фиксированное время
2. Подсчёт событий в скользящем окне времени
3. Атомарная блокировка ресурса для предотвращения конфликтов
4. Атомарное обновление значения при совпадении с ожидаемым
5. Управление очередью задач или сообщений

Подсказка: Думай о цели каждого скрипта: лимиты, блокировки или очереди.

- **Rate Limiting** — Ограничение количества запросов за фиксированное время
- **Sliding Window** — Подсчёт событий в скользящем окне времени
- **Distributed Lock** — Атомарная блокировка ресурса для предотвращения конфликтов
- **Compare-And-Set** — Атомарное обновление значения при совпадении с ожидаемым
- **Queues** — Управление очередью задач или сообщений

 Шаг 05:

Миссия 5

Формулировка: Сопоставьте тип скрипта с его применением.

Ответ:

- Счётчики с TTL
- Сессии и TTL
- Queues
- Compare-And-Set

1. Хранение временных данных с автоматическим истечением срока жизни
2. Контроль активности пользователей и автоматическое завершение сессий
3. Организация очереди задач или событий
4. Атомарное обновление значения при совпадении текущего значения

Подсказка: TTL помогает автоматизировать время жизни данных.

- **Счётчики с TTL** — Хранение временных данных с автоматическим истечением срока жизни
- **Сессии и TTL** — Контроль активности пользователей и автоматическое завершение сессий
- **Queues** — Организация очереди задач или событий
- **Compare-And-Set** — Атомарное обновление значения при совпадении текущего значения

 Шаг 06:

Миссия 6

Формулировка: Сопоставьте Redis-команду с её Lua-эквивалентом.

Ответ:

- INCR
- SETNX

- LPUSH
- EXPIRE

1. Атомарное увеличение числового значения ключа
2. Установить ключ только если он не существует
3. Добавление элемента в начало списка
4. Задание времени жизни ключа

Подсказка: В Lua команды Redis вызываются через `redis.call()`.

- **INCR** — Атомарное увеличение числового значения ключа
- **SETNX** — Установить ключ только если он не существует
- **LPUSH** — Добавление элемента в начало списка
- **EXPIRE** — Задание времени жизни ключа

Шаг 07:

Миссия 7

Вопрос: Расположите шаги создания скрипта для Rate Limiting в правильном порядке:

1. Определение ключа и лимита запросов
2. Проверка текущего значения счётчика
3. Увеличение счётчика и установка TTL
4. Возврат результата клиенту о возможности выполнения действия

Подсказка: Думай о порядке действий: сначала проверка, затем изменение, затем ответ.

Ответ:

- Определение ключа и лимита запросов
- Проверка текущего значения счётчика
- Увеличение счётчика и установка TTL
- Возврат результата клиенту о возможности выполнения действия

Шаг 08:

Миссия 8

Вопрос: Расположите шаги реализации Distributed Lock в правильном порядке:

1. Попытка установить ключ с помощью SETNX
2. Проверка, был ли ключ успешно установлен
3. Исполнение критической секции
4. Удаление ключа после завершения

Подсказка: Думай о безопасной блокировке и освобождении ресурса.

Ответ:

- Попытка установить ключ с помощью SETNX
- Проверка, был ли ключ успешно установлен
- Исполнение критической секции
- Удаление ключа после завершения

Шаг 09:

Миссия 9

Вопрос: Расположите шаги создания скрипта для очередей (Queues) в правильном порядке:

1. Добавление задачи в очередь с помощью LPUSH
2. Обработка задачи из конца списка с помощью RPOP
3. Проверка наличия элементов в очереди
4. Возврат результата обработки задачи

Подсказка: Думай о классическом FIFO или LIFO порядке очередей.

Ответ:

- Добавление задачи в очередь с помощью LPUSH
- Обработка задачи из конца списка с помощью RPOP
- Проверка наличия элементов в очереди
- Возврат результата обработки задачи

Шаг 10:

Миссия 10

Вопрос: Расположите шаги использования Compare-And-Set для атомарного обновления в правильном порядке:

1. Считать текущее значение ключа
2. Сравнить его с ожидаемым значением
3. Если совпадает, записать новое значение
4. Вернуть результат операции клиенту

Подсказка: Думай о проверке перед изменением и сообщении результата.

Ответ:

- Считать текущее значение ключа
- Сравнить его с ожидаемым значением
- Если совпадает, записать новое значение
- Вернуть результат операции клиенту

Модуль 11 - Бонус. Полезные приёмы и рекомендации

 Урок 1 - Всегда проверяйте ошибки при Run в Go

 Урок 2 - Не используйте KEYS в продакшене — лучше использовать SCAN

 Урок 3 - Хранение Lua-скриптов в Go

 Урок 4 - Скрипты атомарны — нет риска гонки данных

 Урок 5 - Квест по мотивам реальной работы программистов — 1

Шаг 01:

Миссия 1

Ты запускаешь Lua-скрипт через Go и хочешь быть уверен, что он отработает корректно. Что обязательно нужно сделать?

Подсказка: Всегда думай о возможных ошибках во время выполнения.

Ответ:

- Проверять ошибки после Run

- Игнорировать ошибки
- Сразу писать лог без проверки
- Вызывать скрипт дважды подряд

🔮 Шаг 02:

Миссия 2

Ты в продакшене и хочешь найти все ключи по шаблону. Как безопаснее всего это сделать?

Подсказка: Подумай о нагрузке на Redis и масштабируемости.

Ответ:

- **Использовать SCAN вместо KEYS ***
- Использовать KEYS * напрямую
- Перезапускать Redis перед поиском
- Удалить все ключи и искать вручную

🔮 Шаг 03:

Миссия 3

Ты хочешь хранить Lua-скрипты в Go и выполнять их динамически. Какой подход правильный?

Подсказка: Подумай о том, чтобы скрипты были доступны и безопасны для вызова.

Ответ:

- **Хранить скрипты в Go и вызывать через EVAL или EVALSHA**
- Вставлять Lua-код прямо в каждый вызов redis.Run
- Сохранять скрипты в текстовый файл и не контролировать доступ
- Использовать только KEYS * для вызова скриптов

🔮 Шаг 04:

Миссия 4

Формулировка: Сопоставьте практику с её преимуществом.

Ответ:

- Проверка ошибок после Run в Go
 - Использование SCAN вместо KEYS *
 - Хранение Lua-скриптов в Go
 - Использование атомарных скриптов
1. Повышение стабильности и отлавливание исключений
 2. Снижение нагрузки на Redis и безопасность продакшена
 3. Удобство управления скриптами и повторное использование
 4. Отсутствие гонок данных при обновлении ключей

Подсказка: Подумай, какое действие защищает от ошибок, а какое улучшает производительность.

- **Проверка ошибок после Run в Go** — Повышение стабильности и отлавливание исключений
- **Использование SCAN вместо KEYS *** — Снижение нагрузки на Redis и безопасность продакшена
- **Хранение Lua-скриптов в Go** — Удобство управления скриптами и повторное использование

- **Использование атомарных скриптов** — Отсутствие гонок данных при обновлении ключей

🔮 Шаг 05:

Миссия 5

Формулировка: Сопоставьте инструмент или команду с её назначением.

Ответ:

- Run в Go
 - SCAN
 - EVALSHA
 - Lua-скрипт атомарный
1. Выполняет Lua-скрипт на Redis через Go
 2. Итерирует по ключам без перегрузки сервера
 3. Выполняет ранее сохранённый скрипт по хешу
 4. Гарантирует отсутствие гонок при обновлении данных

Подсказка: Подумай, какие команды связаны с вызовом скриптов, а какие с безопасностью данных.

- **Run в Go** — Выполняет Lua-скрипт на Redis через Go
- **SCAN** — Итерирует по ключам без перегрузки сервера
- **EVALSHA** — Выполняет ранее сохранённый скрипт по хешу
- **Lua-скрипт атомарный** — Гарантирует отсутствие гонок при обновлении данных

🔮 Шаг 06:

Миссия 6

Формулировка: Сопоставьте ситуацию с правильной рекомендацией.

Ответ:

- Нужно обработать множество ключей в продакшене
 - Хочется, чтобы ошибки Lua-скрипта не приводили к сбою
 - Скрипт нужно вызывать многократно и безопасно
 - Необходимо избежать гонок при обновлении одного ключа
1. Использовать SCAN вместо KEYS *
 2. Проверять ошибки после Run
 3. Хранить скрипт в Go и вызывать через EVALSHA
 4. Делать скрипт атомарным

Подсказка: Смотри на цель каждого действия: безопасность, повторное использование или атомарность.

- **Нужно обработать множество ключей в продакшене** — Использовать SCAN вместо KEYS *
- **Хочется, чтобы ошибки Lua-скрипта не приводили к сбою** — Проверять ошибки после Run
- **Скрипт нужно вызывать многократно и безопасно** — Хранить скрипт в Go и вызывать через EVALSHA
- **Необходимо избежать гонок при обновлении одного ключа** — Делать скрипт атомарным

Урок 6 - Квест по мотивам реальной работы программистов — 2

Шаг 01:

Миссия 1

Ты запускаешь Lua-скрипт через Go. Что важно сделать после выполнения `redis.Run`?

Подсказка: Любая операция может завершиться ошибкой, не забывай проверять результат.

Ответ:

- **Проверить ошибки выполнения**
- Игнорировать результат
- Всегда закрывать соединение после `Run`
- Сразу перезапускать скрипт без проверки

Шаг 02:

Миссия 2

Почему в продакшене не стоит использовать команду `KEYS *`?

Подсказка: Думай о влиянии на производительность Redis при большом количестве ключей.

Ответ:

- **Она блокирует сервер и может сильно замедлить работу**
- Она не возвращает все ключи
- Она может удалить все ключи случайно
- Она создаёт копию данных

Шаг 03:

Миссия 3

Как безопасно получать все ключи в продакшене?

Подсказка: Подумай о методе, который возвращает ключи порционно и не блокирует сервер.

Ответ:

- **Использовать `SCAN`**
- Использовать `KEYS *`
- Использовать `GETALL`
- Использовать `MGET`

Шаг 04:

Миссия 4

Сопоставьте практику с её объяснением:

Ответ:

- Всегда проверяйте ошибки при `Run` в Go
 - Не используйте `KEYS *` в продакшене
 - Хранение Lua-скриптов в Go
 - Скрипты атомарны
1. Обеспечивает безопасное выполнение операций и отлов ошибок
 2. Использовать `SCAN` вместо, чтобы не блокировать сервер
 3. Удобно для повторного вызова и интеграции
 4. Нет риска гонки данных при обновлениях

Подсказка: Думай о безопасности, производительности и удобстве кода.

- **Всегда проверяйте ошибки при Run в Go** — Обеспечивает безопасное выполнение операций и отлов ошибок
- **Не используйте KEYS * в продакшене** — Использовать SCAN вместо, чтобы не блокировать сервер
- **Хранение Lua-скриптов в Go** — Удобно для повторного вызова и интеграции
- **Скрипты атомарны** — Нет риска гонки данных при обновлениях

Шаг 05:

Миссия 5

Сопоставьте термин с примером:

Ответ:

- Run с проверкой ошибок
 - SCAN вместо KEYS *
 - Lua-скрипт в Go
 - Атомарный скрипт
1. Обработка ошибки redis.Run и логирование
 2. Перебор ключей порционно, чтобы не блокировать сервер
 3. Скрипт хранится как строка или файл для повторного вызова
 4. Изменение значения только если предыдущее совпадает с ожиданием

Подсказка: Думай о практических примерах кода и безопасной работе Redis.

- **Run с проверкой ошибок** — Обработка ошибки redis.Run и логирование
- **SCAN вместо KEYS *** — Перебор ключей порционно, чтобы не блокировать сервер
- **Lua-скрипт в Go** — Скрипт хранится как строка или файл для повторного вызова
- **Атомарный скрипт** — Изменение значения только если предыдущее совпадает с ожиданием

Шаг 06:

Миссия 6

Расположите шаги безопасного вызова Lua-скрипта в Go:

1. Подготовить Lua-скрипт как строку или файл
2. Вызвать redis.Run с параметрами
3. Проверить ошибки выполнения
4. Обработать результат скрипта

Подсказка: Думай о порядке: подготовка → вызов → проверка → обработка.

Ответ:

- Подготовить Lua-скрипт как строку или файл
- Вызвать redis.Run с параметрами
- Проверить ошибки выполнения
- Обработать результат скрипта

Шаг 07:

Миссия 7

Расположите шаги безопасного перебора ключей без KEYS *:

1. Вызвать команду SCAN с курсором
2. Обработать полученные ключи
3. Продолжить перебор до конца курсора
4. Завершить цикл после обработки всех ключей

Подсказка: Думай о безопасном итеративном обходе ключей.

Ответ:

- Вызвать команду SCAN с курсором
- Обработать полученные ключи
- Продолжить перебор до конца курсора
- Завершить цикл после обработки всех ключей